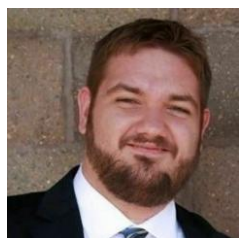


Optimizing Experiments with Pyomo.DoE

<https://dowlinglab.github.io/pyomo-doe/>



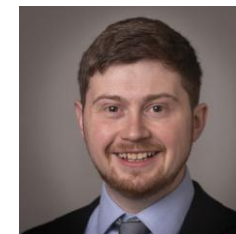
Daniel Laky^{1,2}, Shammah Lilonfe¹, Stephen Cini¹, Shuvashish Mondal¹, Shilpa Narasimhan¹, and Alexander Dowling¹

¹Chemical and Biomolecular Engineering,
University of Notre Dame, Notre Dame, IN

²Chemical Engineering,
Auburn University, Auburn, AL

American Control Conference

May 26, 2026



Other Contributors to ParmEst & Pyomo.DoE: ND: Jialu Wang, Hailey Lynch;
SNL: John Sirola, Bethany Nicholson, Miranda Mundt, Katherine Klise, Shawn Martin

Agenda

Part 1 (12:15pm – 1:30pm): Welcome to Pyomo

- TCLab Mathematical Model
- Simulation and Dynamic Optimization in Pyomo
- Experiment Abstraction

Part 2 (1:45pm – 3pm): Parameter Estimation

- ParmEst Overview
- Uncertainty Quantification
- Multistart & Profile Likelihood
- Regularization

Part 3 (3:30pm – 5pm): Optimal Experiment Design

- Pyomo.DoE Overview
- OED Objectives
- Symbolic Derivatives
- Planning Multiple Experiments



Power of Adaptive Sequential Optimal Experiments

Self-Driving Laboratories

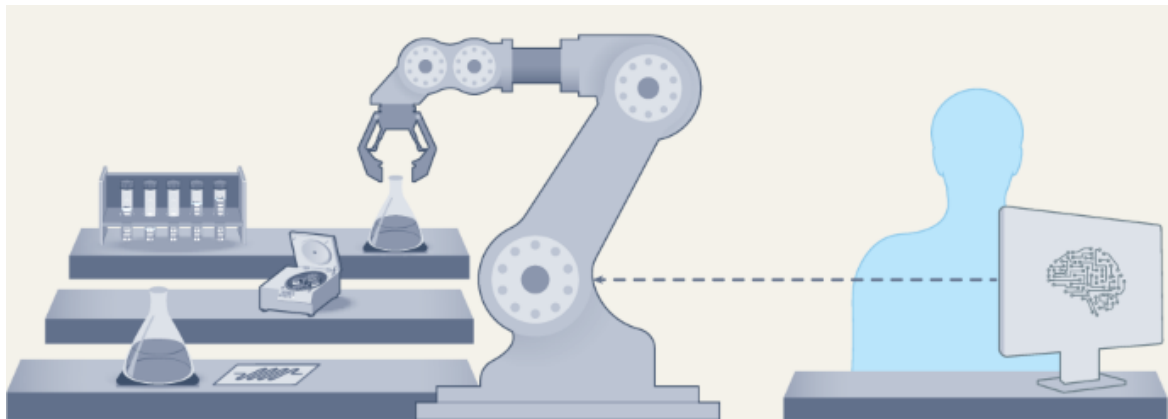


Figure: Abolhasani & Kumacheva (2023), *Nature Syn.*

Epps et al. (2022), *Advanced Materials*

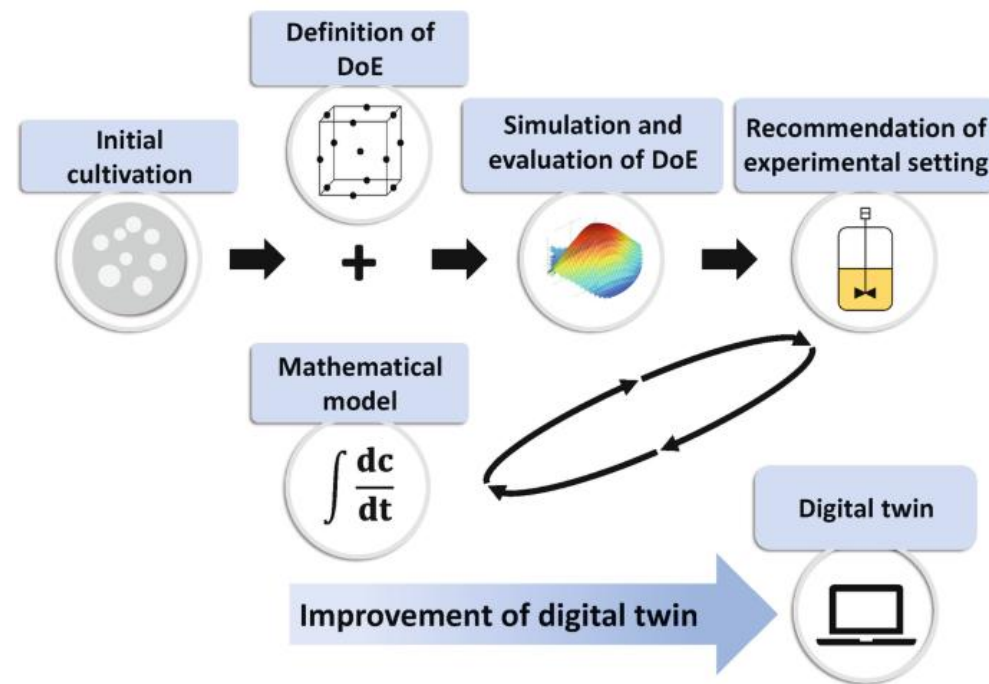
MacLeod et al. (2020), *Science Advances*

MacLeod et al. (2022), *Nature Communications*

Hase, Roch, Aspuru-Guzik (2019), *Trends in Chemistry*

Seifrid et al. (2022), *Acc. Chem. Res.*

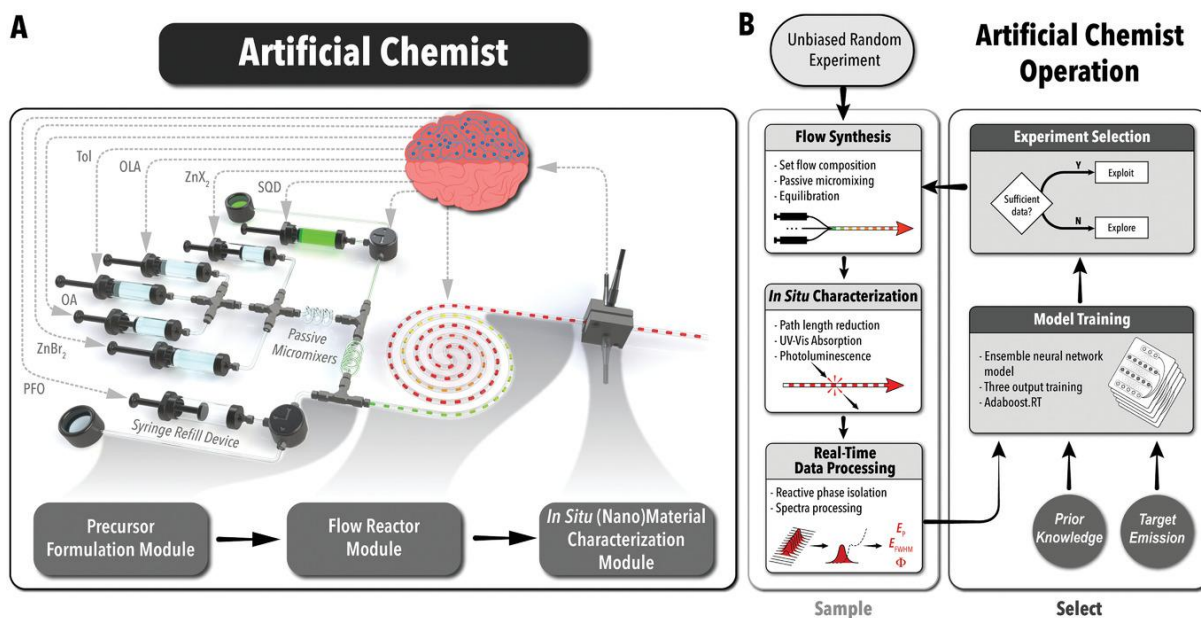
Automation + Model-Based Design of Experiments



Kuchemuller et al. (2020), *Digital Twins*

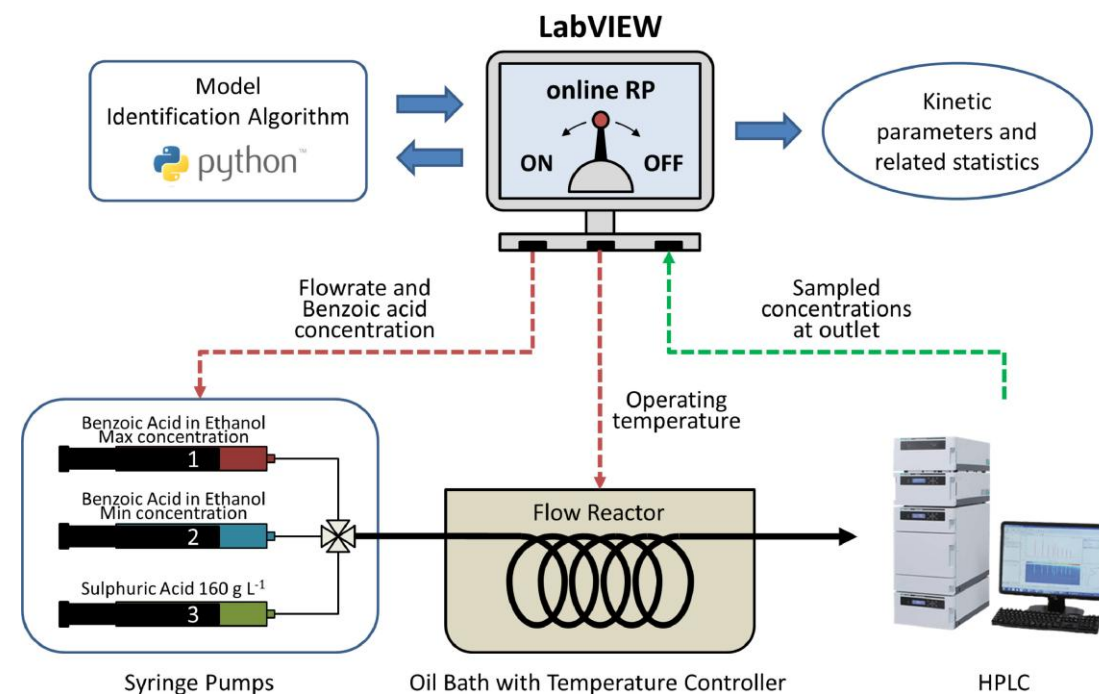
Many Recent Examples of Sequential Optimal Experiments

Quantum Dots (Machine Learning)



Epps et al. (2022), *Advanced Materials*

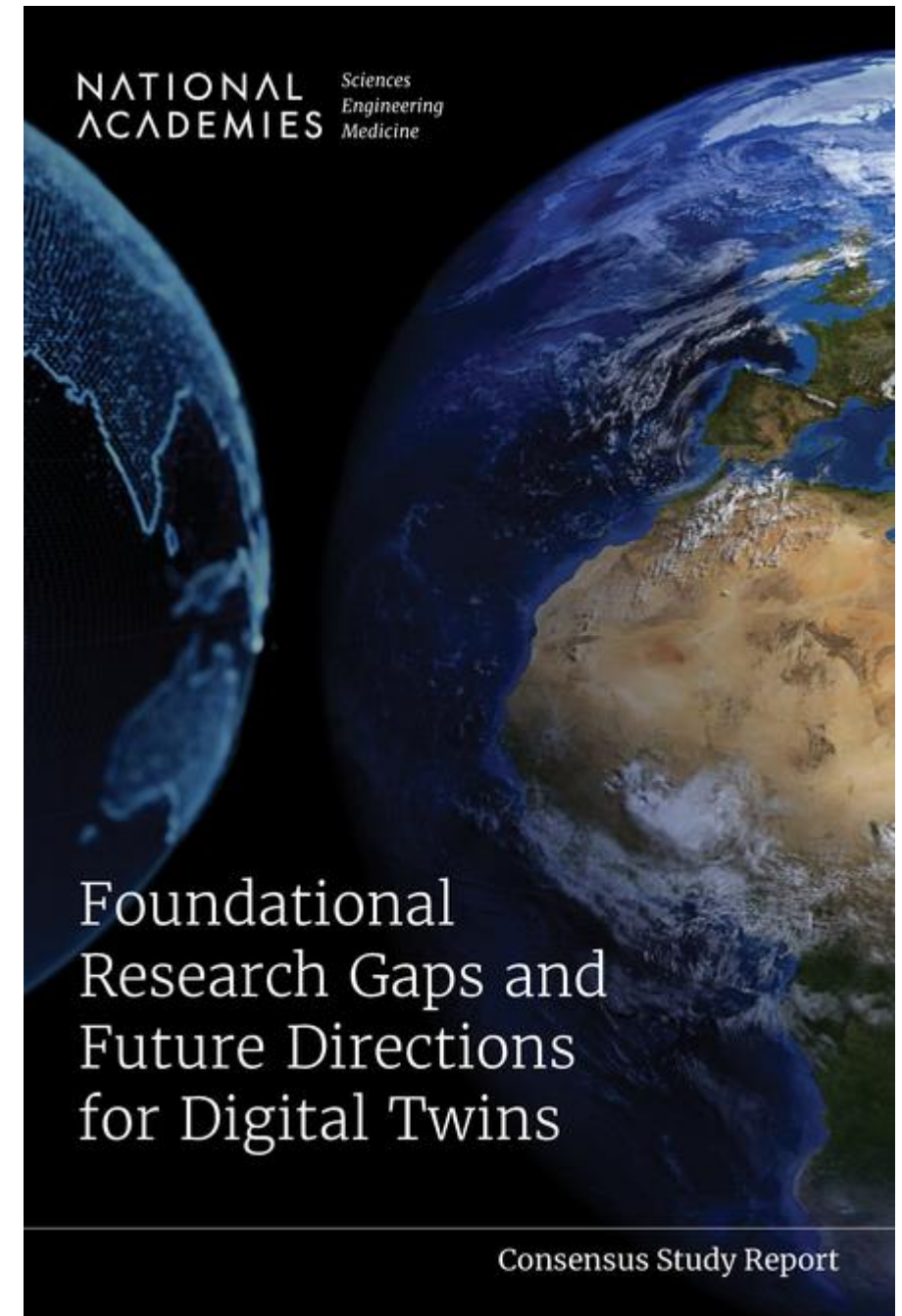
Reaction Engineering (Science-based Models)



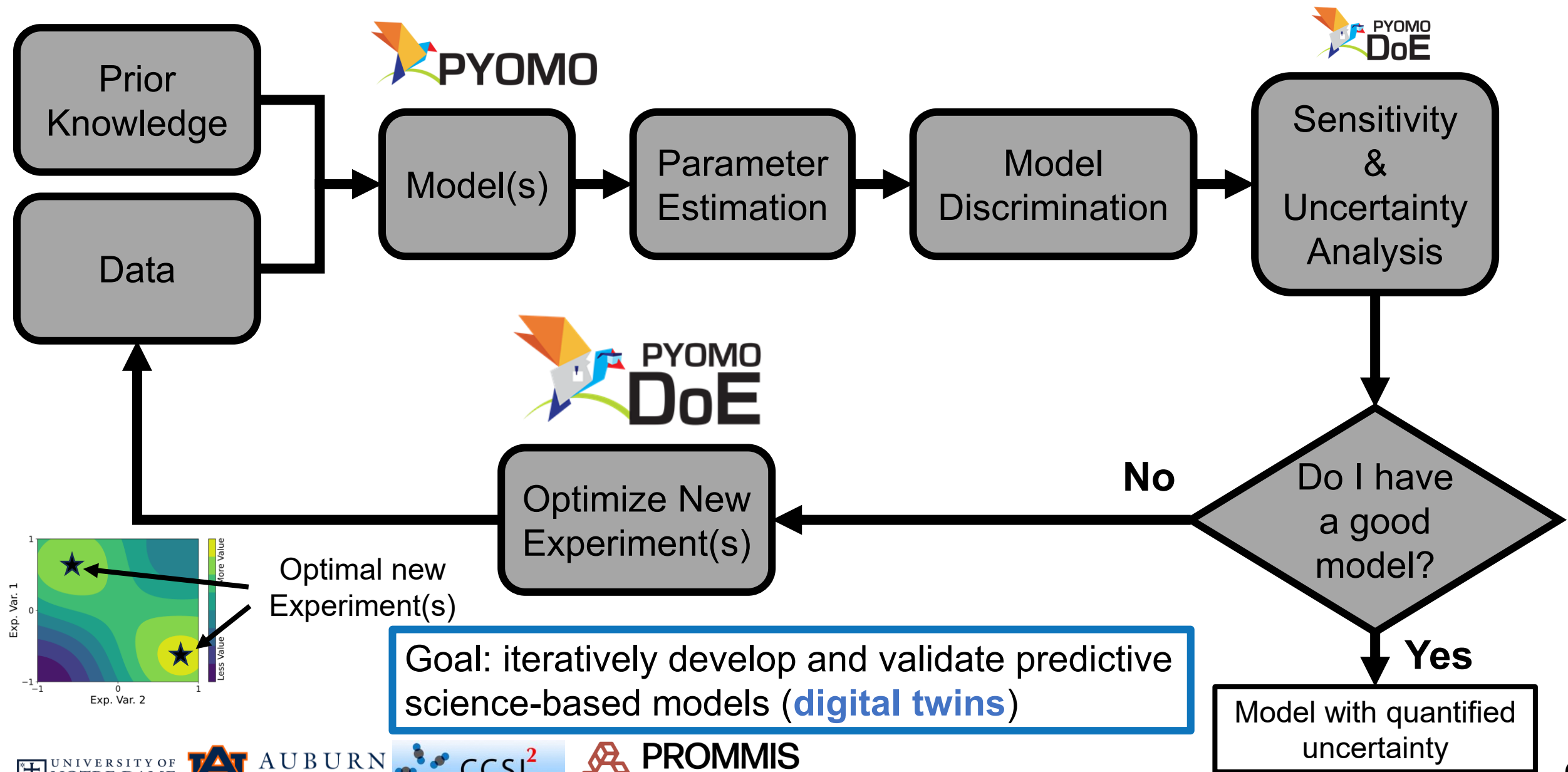
Quaglio et al. (2019), *Comp. & Chem. Eng.*

What is a **Digital Twin**?

*A **digital twin** is a set of **virtual information constructs** that mimics the structure, context, and behavior of a **natural, engineered, or social system** (or **system-of-systems**), is **dynamically updated** with data from its physical twin, has a **predictive capability**, and **informs decisions** that **realize value**. The **bidirectional interaction** between the virtual and the physical is central to the digital twin.*

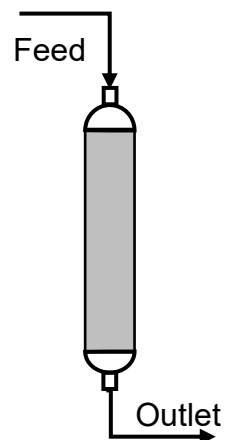


Science-based Data Analytics Workflow

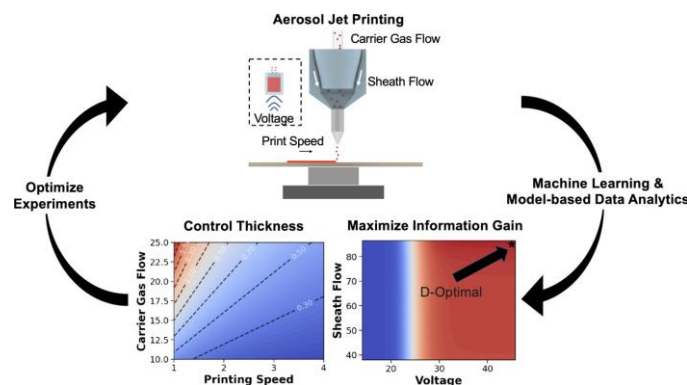


SBDoE (a.k.a. MBDoE) Facilitates Collaborations

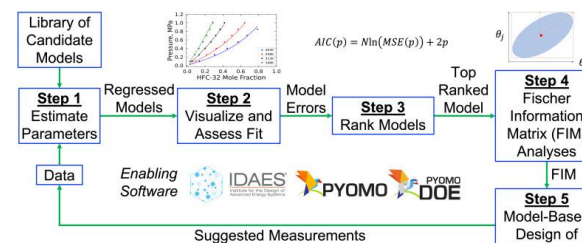
CO₂ Capture



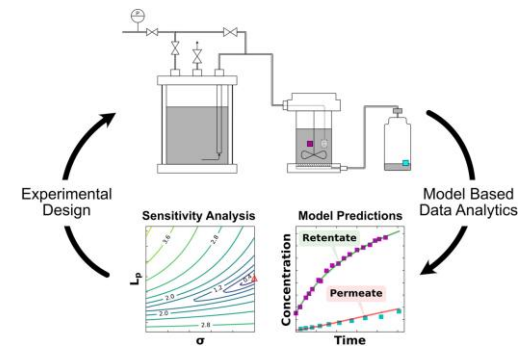
Additive Manufacturing of Thermoelectric Devices



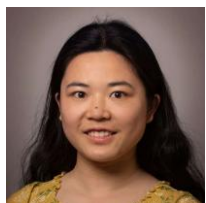
Thermodynamic Modeling (Refrigerants)



Rapid/Automated Membrane Characterization



Dr. Jialu Wang



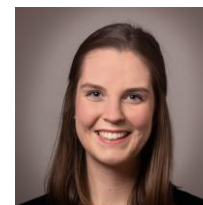
Wang, J. and Dowling, A.W. (2022), *AIChE J.* e17813.

Dr. Ke Wang



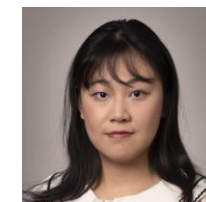
Wang K., Zhang M., Wang, J., Shang, W., Zhang, Y., Luo, T., Dowling, A.W. (2023), *Digital Chemical Engineering*

Dr. Bridgette Befort



Befort, B.J., Garciadiego, A., Wang, J., Wang, K., Maginn, E.J., Dowling, A.W. (2023), *Fluid Phase Equilibria*.

Xinhong Liu



Ouimet, J.A, Xinhong, L., Brown, D.J., Eugene, E.A., Popp, T., Muetzel, Z.W., Dowling, A.W., Phillip, W.A., (2022). *J. Membrane Science*.

Agenda

Part 1 (12:15pm – 1:30pm): Welcome to Pyomo

- TCLab Mathematical Model
- Simulation and Dynamic Optimization in Pyomo
- Experiment Abstraction

Part 2 (1:45pm – 3pm): Parameter Estimation

- ParmEst Overview
- Uncertainty Quantification
- Multistart & Profile Likelihood
- Regularization

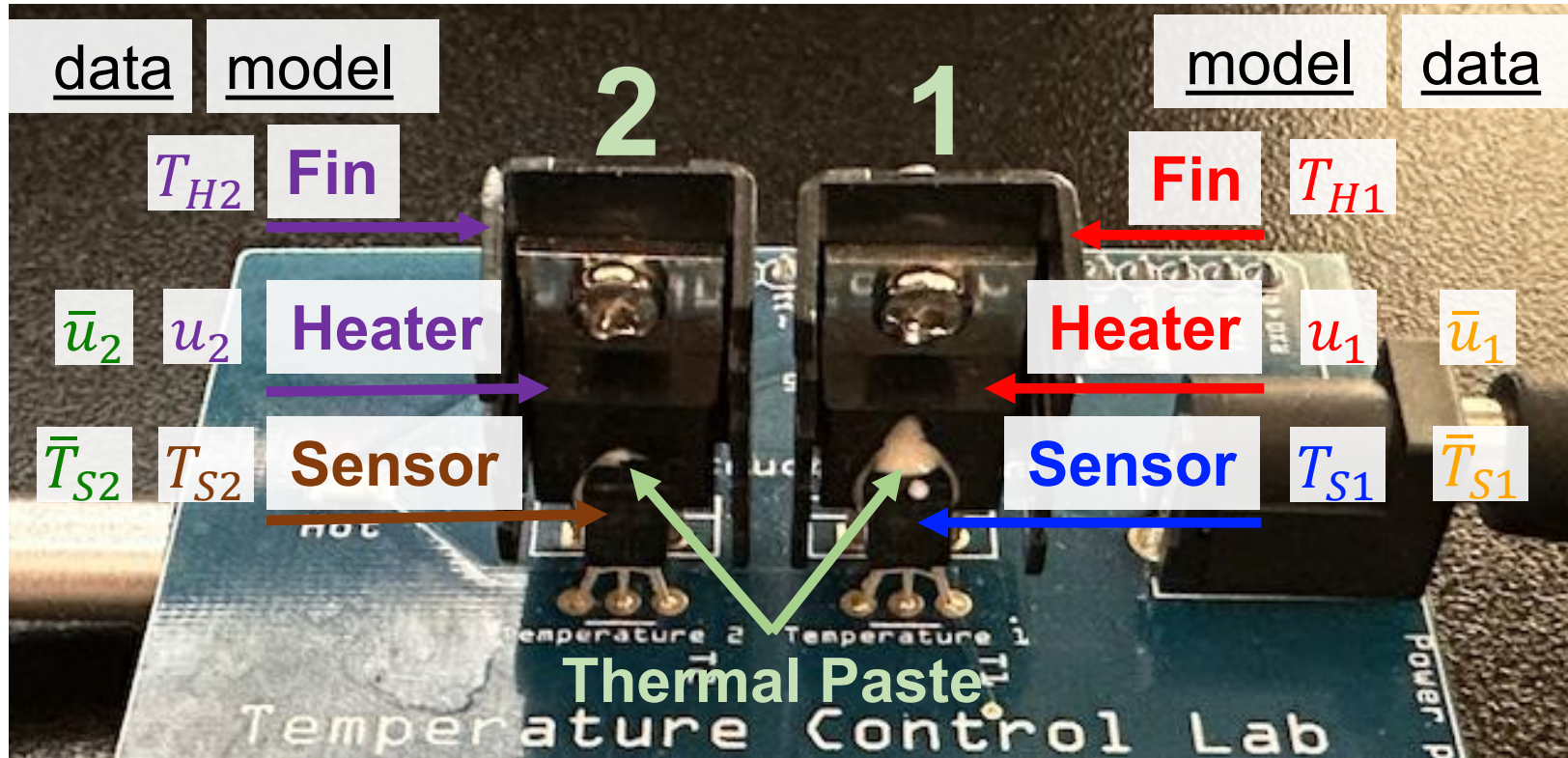
Part 3 (3:30pm – 5pm): Optimal Experiment Design

- Pyomo.DoE Overview
- OED Objectives
- Symbolic Derivatives
- Planning Multiple Experiments



The Temperature Control Lab (TC Lab)

<https://ndcbe.github.io/controls>



Dowling et al. (2025). *Systems and Control Transactions*



Thank you to Prof. Jeff Kantor (1954-2023) for the TCLab example and so much more.

TC Lab single heater model: two linear differential equations

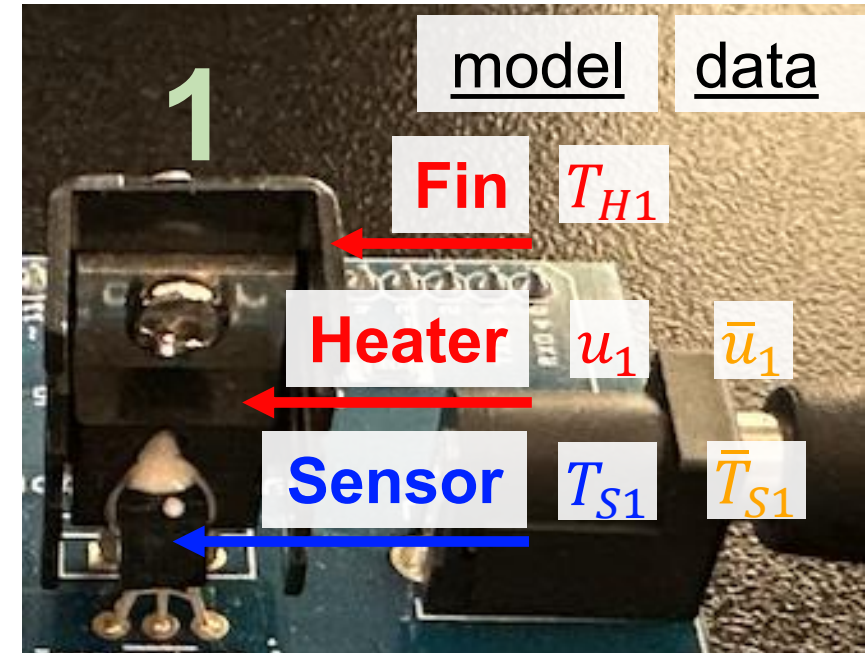
Dowling et al. (2025). *Systems and Control Transactions*

Energy balance on the heater

$$\underbrace{C_{p,H} \frac{dT_{H,1}}{dt}}_{\text{Accumulation}} = \underbrace{\alpha P_1 u_1}_{\text{Input}} - \underbrace{\left(U_a (T_{H,1} - T_{amb}) + U_b (T_{H,1} - T_{S,1}) \right)}_{\text{Output}}$$

Energy balance on the sensor assuming no heat transfer to the ambient

$$\underbrace{C_{p,S} \frac{dT_{S,1}}{dt}}_{\text{Accumulation}} = \underbrace{U_b (T_{H,1} - T_{S,1})}_{\text{Input}}$$



U_a = heater heat transfer coefficient in W/°C,
 U_b = sensor heat transfer coefficient in W/°C,
 $C_{p,H}$ = heat capacity of the heater in J/°C, $C_{p,S}$ = heat capacity of the sensor in J/°C,
 $T_{H,1}$ = temperature of heater 1 in °C, $T_{S,1}$ = temperature of sensor 1 in °C,
 T_{amb} = ambient temperature in °C, t = time in s,
 u_1 = heater 1 power in %, P_1 = hardware-set maximum power limit in bit,
 α = parameter for converting the unit of $P_1 u_1$ into W,

Accumulation

Input

TC Lab single heater model: two linear differential equations

Model from energy balance

$$C_{p,H} \frac{dT_{H,1}}{dt} = \alpha P_1 u_1 - \left(U_a (T_{H,1} - T_{\text{amb}}) + U_b (T_{H,1} - T_{S,1}) \right), \quad C_{p,S} \frac{dT_{S,1}}{dt} = U_b (T_{H,1} - T_{S,1})$$

Rearranged TC Lab model

$$\frac{dT_{H,1}}{dt} = \frac{U_a (T_{\text{amb}} - T_{H,1}) + U_b (T_{S,1} - T_{H,1}) + \alpha P_1 u_1}{C_{p,H}}, \quad \frac{dT_{S,1}}{dt} = \frac{U_b (T_{H,1} - T_{S,1})}{C_{p,S}}$$

Unknown parameters

$$\theta = \left(U_a, U_b, (C_{p,H})^{-1}, (C_{p,S})^{-1} \right)^T$$

Measured variables

$$u_1, T_{S,1}$$

Control variables

$$u_1$$

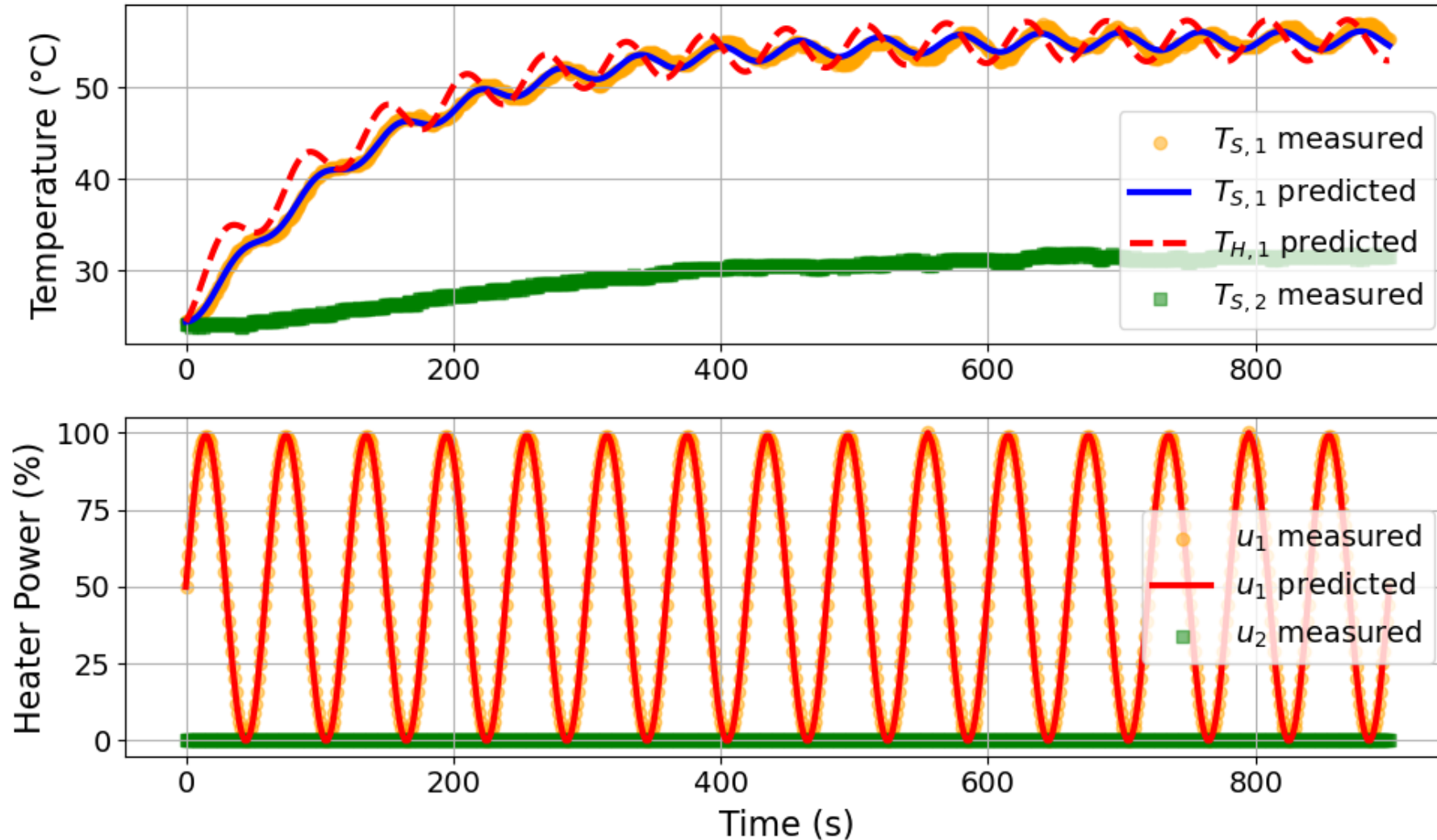
Data

$$T_{\text{amb}}, \alpha, P_1, u_1, T_{S,1}$$

U_a = heater heat transfer coefficient in W/°C,
 U_b = sensor heat transfer coefficient in W/°C,
 $C_{p,H}$ = heat capacity of the heater in J/°C, $C_{p,S}$ = heat capacity of the sensor in J/°C,
 $T_{H,1}$ = temperature of heater 1 in °C, $T_{S,1}$ = temperature of sensor 1 in °C,
 T_{amb} = ambient temperature in °C, t = time in s,
 u_1 = heater 1 power in %, P_1 = hardware-set maximum power limit in bit,
 α = parameter for converting the unit of $P_1 u_1$ into W,

TC Lab: Data and Parameter Estimation

Hands-On Tutorial: dowlinglab.github.io/pyomo-doe



Notebooks at `dowlinglab.github.io/pyomo-doe`



CONTENTS ▾

TCLab Mathematical Model

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal,
Shilpa Narasimhan

We will use the TCLab as a motivating example for this workshop.

The code below sets default font sizes.

```
import matplotlib.pyplot as plt

SMALL_SIZE = 14
MEDIUM_SIZE = 16
BIGGER_SIZE = 18
```

Temperature Control Lab

System Identification Data

Step Test

Sine Test

Two-State Mathematical Model

State Space Model

Notebooks at `dowlinglab.github.io/pyomo-doe`



CONTENTS ▾

Modeling with Pyomo

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal, Shilpa Narasimhan

This page is [adapted from our process control class](#) at Notre Dame; it was developed by Prof. Jeff Kantor.

```
# Install Pyomo and solvers for Google Colab
import sys

if "google.colab" in sys.modules:
    !wget "https://raw.githubusercontent.com/IDAES/idaes-pse/main/scripts/colab_helper.py"
    import colab_helper

    colab_helper.install_idaes()
    colab_helper.install_ipopt()

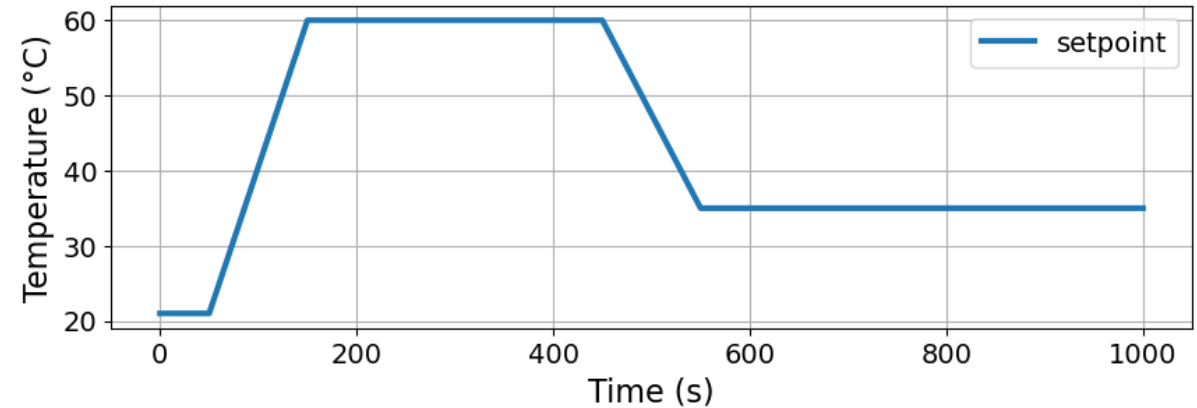
# Set plotting defaults
import matplotlib.pyplot as plt
```

- Simulate a Step Test (Ramp)
- Activity: Optimization by Trial-and-Error
- Feedforward Optimal Control
- Controlling to a Reference Trajectory
- Helper Functions
- Take Away Messages

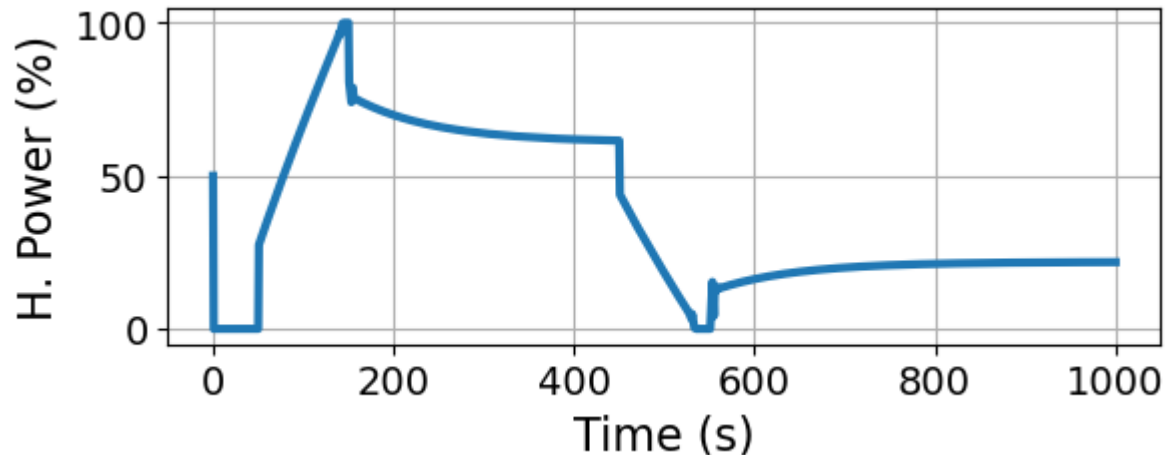
TC Lab: Dynamic Optimization in Pyomo

dowlinglab.github.io/pyomo-doe/notebooks/pyomo_simulation.html

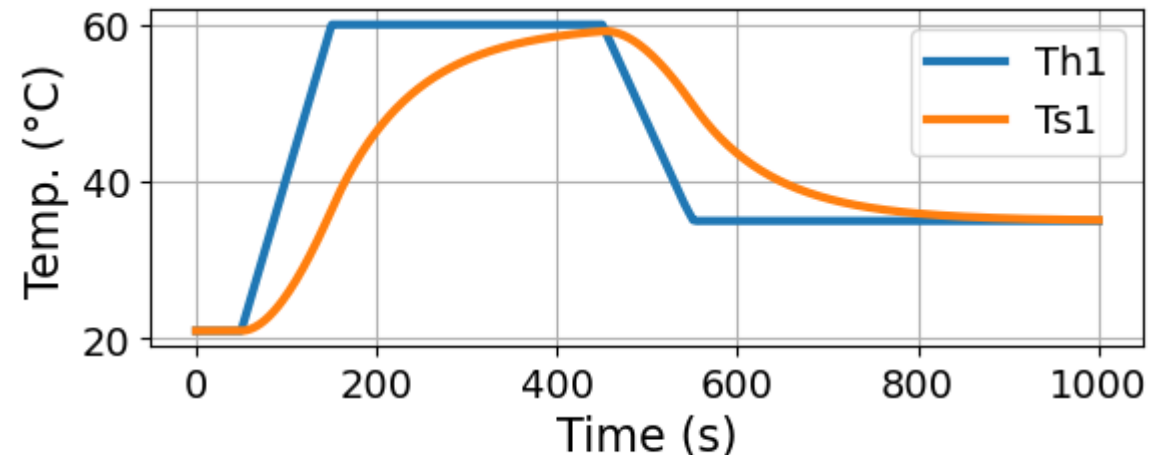
$$\begin{aligned} \min_{u(t)} \quad & \int_{t_0}^{t_f} \| SP(t) - T_H(t) \|^2 dt \\ \text{s. t.} \quad & C_p^H \frac{dT_H}{dt} = U_a(T_{amb} - T_H) + U_b(T_S - T_H) + \alpha P u(t) \\ & C_p^S \frac{dT_S}{dt} = U_b(T_H - T_S) \\ & T_H(t_0) = T_{amb} \\ & T_S(t_0) = T_{amb} \end{aligned}$$



Optimal control profile



T_{h1} matches the setpoint using optimal control



TCLab & Process Controls UG Course at Notre Dame

Systems & Control
Transactions

Research Article - Peer Reviewed Conference Proceeding
ESCAPE 35 - European Symposium on Computer Aided Process Engineering
Ghent, Belgium. 6-9 July 2025
Jan F.M. Van Impe, Grégoire Léonard, Satyajeet S. Bhonsale,
Monika E. Polańska, Filip Logist (Eds.)



Teaching Digital Twins in Process Control Using the Temperature Control Lab

Alexander W. Dowling*, Molly Dougher, Madelynn J. Watson, Hailey G. Lynch, Zhicheng Lu, and Daniel J. Laky

University of Notre Dame, Department of Chemical and Biomolecular Engineering, Notre Dame, IN 46556, United States

* Corresponding Author: adowling@nd.edu.

ABSTRACT

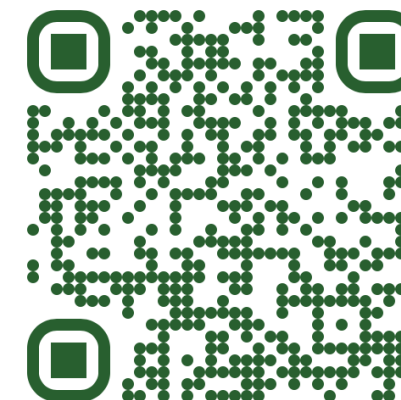
Process control can be one of the most exciting and engaging chemical engineering undergraduate courses! This paper describes our experience transforming *Chemical Process Control* into *Data Analytics, Optimization, and Control* at the University of Notre Dame (second semester required course in the junior year). Our modern course is built around six hands-on experiments in which students practice data-centric modeling and analysis using the Arduino-based Temperature Control Lab (TCLab) hardware. We argue that state-space dynamic modeling and optimization are more critical for educating modern chemical engineers than topics such as frequency domain analysis and controller synthesis emphasized in many classical undergraduate control courses. All the course material is available online at <https://ndcbe.github.io/controls>.

Keywords: Process Control, Education, Dynamic Modelling, System Identification, Model Predictive Control, Pyomo, Process Monitoring, Process Operations, Industry 4.0

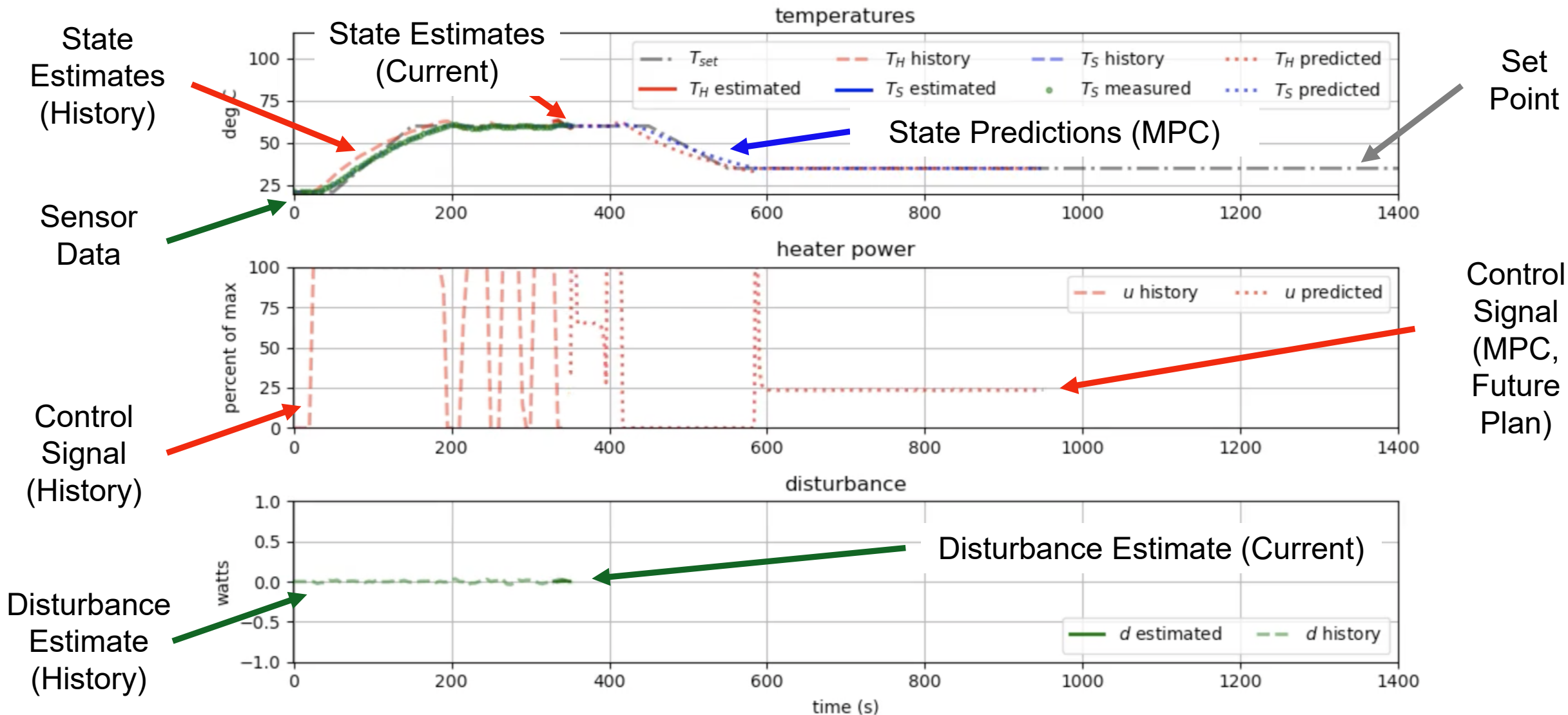
Course Materials



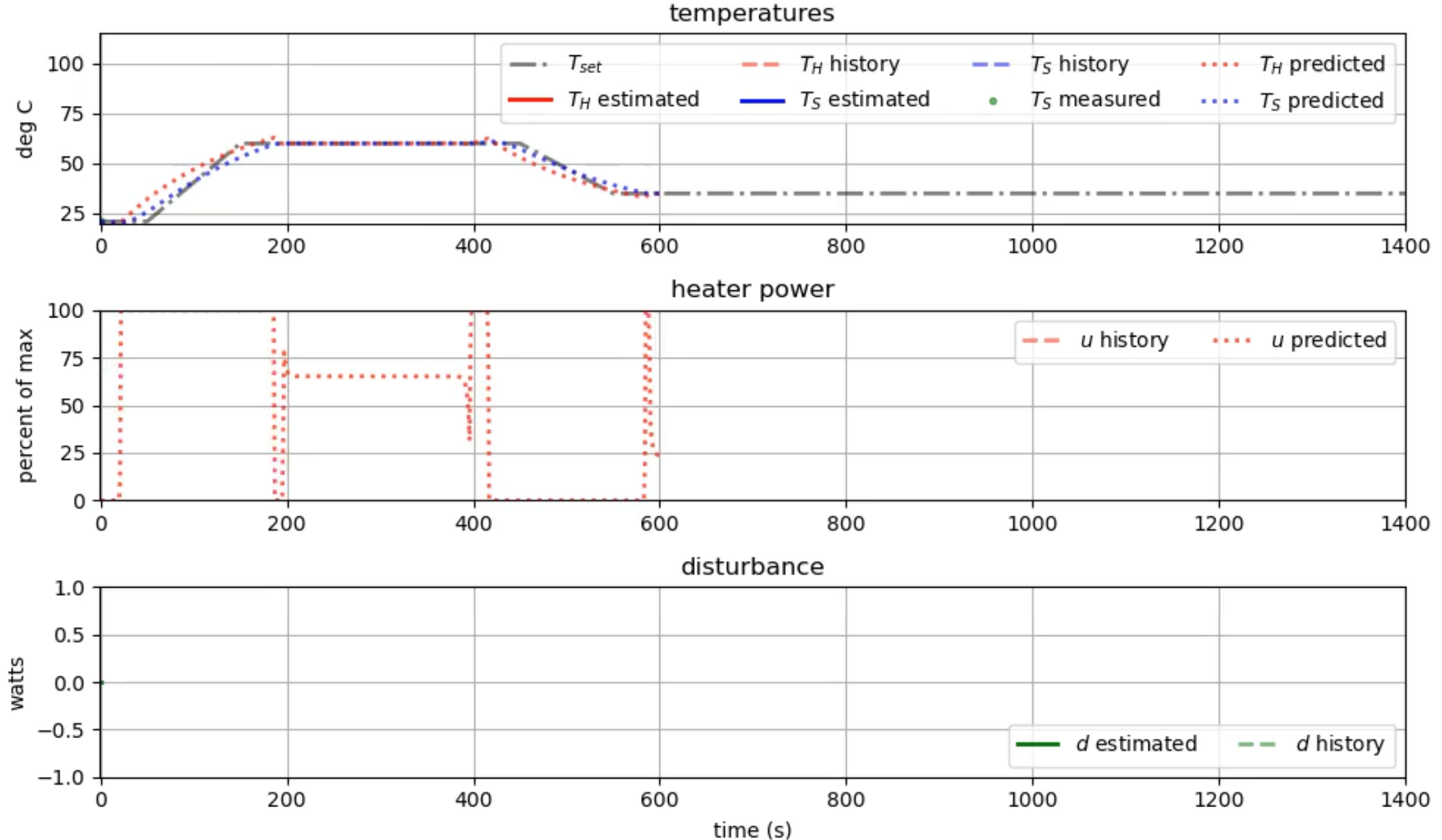
Paper



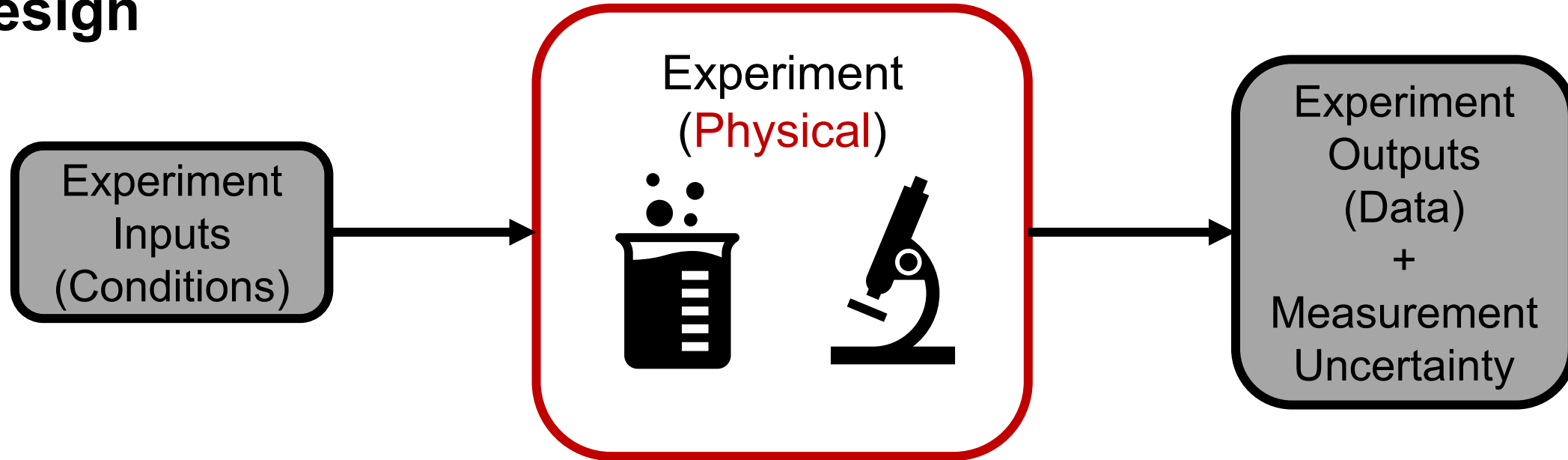
Teaching Digital Twins (MPC, State Estimation)



Teaching Digital Twins (MPC, State Estimation)



“Experiment” Abstraction Streamlines Closed-Loop Experiment Design



Dr. Bethany Nicholson



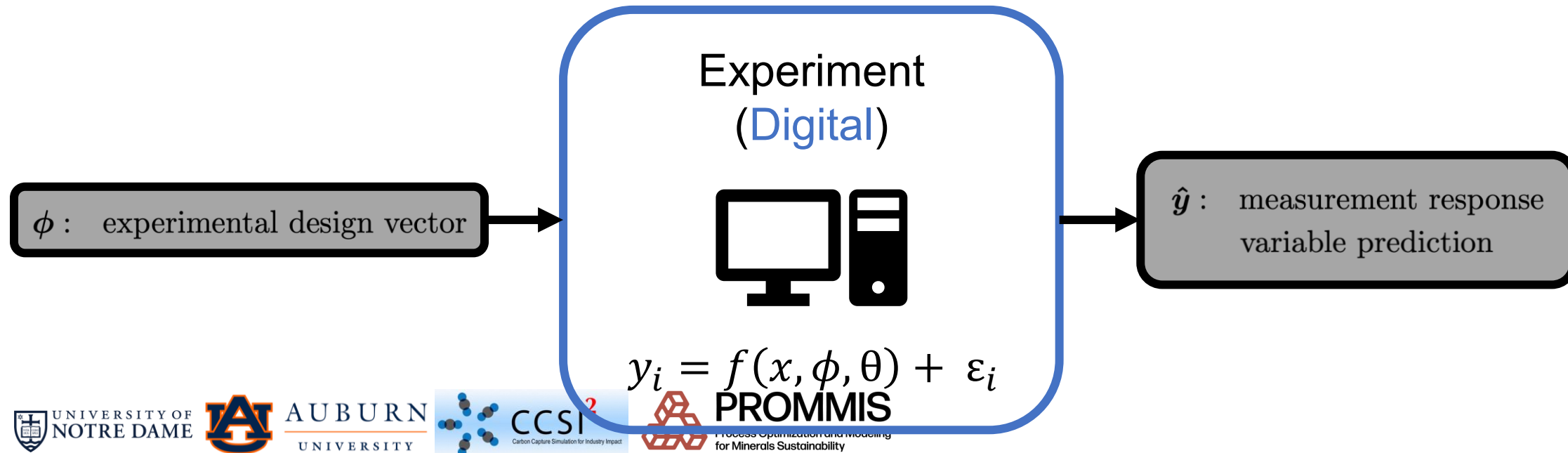
Dr. John Siirola



Dr. Shawn Martin



Katherine Klise



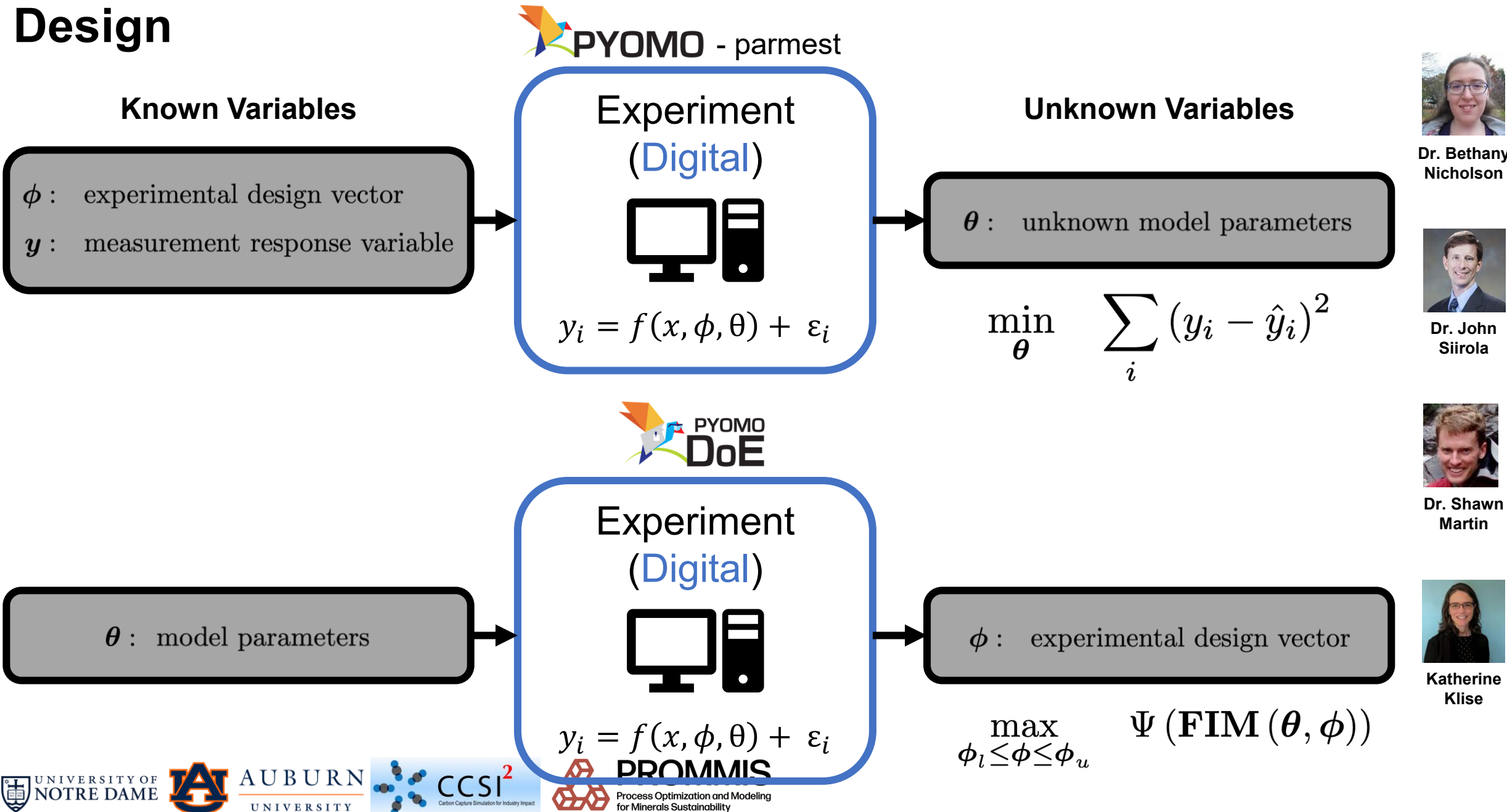
$$y_i = f(x, \phi, \theta) + \epsilon_i$$

PROMMIS

Process Optimization and Modeling
for Minerals Sustainability



“Experiment” Abstraction Streamlines Closed-Loop Experiment Design



Notebooks at `dowlinglab.github.io/pyomo-doe`

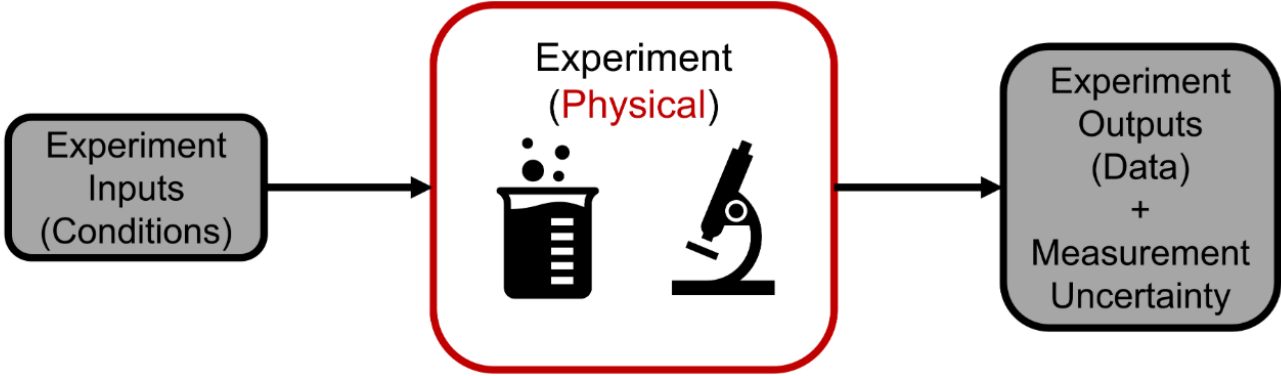


CONTENTS ▾

Experiment Abstraction

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal, Shilpa Narasimhan

The fundamental basis to parameter estimation and to optimal, science-based design of experiments (SBDoe) is an experiment. This concept is also borrows from the concept of a physical experiment:



```
graph LR; A[Experiment Inputs (Conditions)] --> B[Experiment (Physical)]; B --> C[Experiment Outputs (Data) + Measurement Uncertainty];
```

Agenda

Part 1 (12:15pm – 1:30pm): Welcome to Pyomo

- TCLab Mathematical Model
- Simulation and Dynamic Optimization in Pyomo
- Experiment Abstraction

Part 2 (1:45pm – 3pm): Parameter Estimation

- ParmEst Overview
- Uncertainty Quantification
- Multistart & Profile Likelihood
- Regularization

Part 3 (3:30pm – 5pm): Optimal Experiment Design

- Pyomo.DoE Overview
- OED Objectives
- Symbolic Derivatives
- Planning Multiple Experiments



Notebooks at `dowlinglab.github.io/pyomo-doe`



CONTENTS ▾

Uncertainty Quantification

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal,
Shilpa Narasimhan

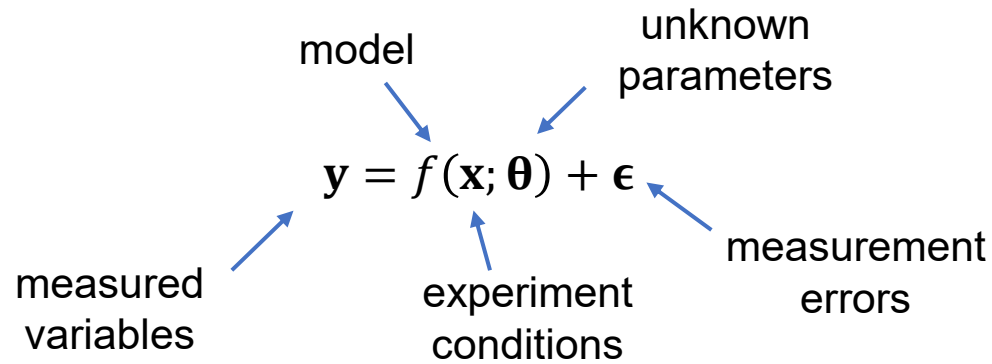
Uncertainty quantification is required to ensure that the estimates of the parameters are close to their true values. This parameter accuracy is measured by computing the covariance matrix. The leading diagonal of this matrix contains the variance of the estimated parameters. The uncertainty in the parameters is taken as the standard deviation of the variance from the covariance matrix.

Assuming Gaussian (correlated or independent and identically distributed) measurement errors, the covariance matrix can be computed using three methods in ParmEst.

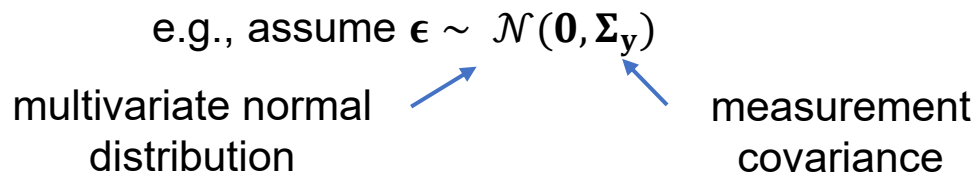
- Computing the Covariance Matrix with ParmEst
 - Import the Necessary Packages
 - Load the Experimental Data
 - Get the Parameter Estimates
 - Covariance Matrix via Reduced Hessian
 - Covariance Matrix via Finite Difference
 - Covariance Matrix via Automatic Differentiation
 - Compare the Covariance Matrix Results
 - Takeaways from the Covariance Matrix Results

Parameter estimation background

1. Express the data in the mathematical form:



2. Define measurement error model:



3. Construct the (log) likelihood function

$$l(\boldsymbol{\theta}; y_1 \cdots y_n) = \underbrace{-\frac{n}{2} (\log(2\pi) + \log|\det \boldsymbol{\Sigma}_y|)}_{\text{constant } c \text{ when } \boldsymbol{\Sigma}_y \text{ is known}} - \underbrace{\frac{1}{2} \sum_{i=1}^n (\mathbf{y}_i - \mathbf{f}(\mathbf{x}_i; \boldsymbol{\theta}))^T \boldsymbol{\Sigma}_y^{-1} (\mathbf{y}_i - \mathbf{f}(\mathbf{x}_i; \boldsymbol{\theta}))}_{\text{WSSE}}$$

4. Compute the maximum likelihood estimator

maximum likelihood estimate

$$\hat{\boldsymbol{\theta}} = \arg \max_{\boldsymbol{\theta}} l(\boldsymbol{\theta}) = \arg \min_{\boldsymbol{\theta}} \text{WSSE}$$

y = measured variables, ϵ = measurement errors, l = log-likelihood function, n = number of experiments, $\boldsymbol{\theta}$ = parameters, WSSE = weighted sum of squared errors, $\mathbf{x}$ = experimental conditions, $\boldsymbol{\Sigma}_y$ = measurement error covariance matrix

Covariance matrix quantifies parameter uncertainty

5. Define the Fisher information matrix:

$$\mathbf{FIM}(\boldsymbol{\theta}) = -\mathbb{E}_{\boldsymbol{\theta}} \left[\frac{\partial^2 l(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^2} \right]$$

Information about parameters $\boldsymbol{\theta}$ contained in measured variable $\mathbf{y}$ from experiment design $\mathbf{x}$

6. Define the parameter covariance matrix:

For a consistent and asymptotically normal estimator: $\text{Cov}(\hat{\boldsymbol{\theta}}) = \mathbf{FIM}^{-1}$

7. Compute the parameter covariance matrix:

With $l(\boldsymbol{\theta}) = -\text{WSSE} + c$: $\mathbf{V}_{\boldsymbol{\theta}} \approx -\left(\frac{\partial^2 l(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^2} \right)_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}^{-1}$

Method 1: *reduced_hessian*

$$\mathbf{V}_{\boldsymbol{\theta}} = \left(\frac{\partial^2 \text{WSSE}}{\partial \boldsymbol{\theta}^2} \right)_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}^{-1}$$

Computes reduced Hessian via PyNumero

Method 2: *finite_difference*

$$\frac{\partial^2 \text{WSSE}}{\partial \boldsymbol{\theta}^2} \approx \sum_{i=1}^n \left(\frac{\partial f(\mathbf{x}_i; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \right)^T \boldsymbol{\Sigma}_y^{-1} \left(\frac{\partial f(\mathbf{x}_i; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \right)$$

$$\mathbf{V}_{\boldsymbol{\theta}} = \left(\sum_{i=1}^n \mathbf{Q}_i^T \boldsymbol{\Sigma}_y^{-1} \mathbf{Q}_i \right)_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}^{-1}$$

Uses central finite difference to compute $\mathbf{Q}_i$

Method 3: *automatic_differentiation_kaug*

$$\mathbf{Q}_{\text{kaug}, i} = \frac{\partial f(\mathbf{x}_i; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} + \frac{\partial f(\mathbf{x}_i; \boldsymbol{\theta})}{\partial \mathbf{x}_i} \frac{\partial \mathbf{x}_i}{\partial \boldsymbol{\theta}}$$

$$\mathbf{V}_{\boldsymbol{\theta}} = \left(\sum_{i=1}^n \mathbf{Q}_{\text{kaug}, i}^T \boldsymbol{\Sigma}_y^{-1} \mathbf{Q}_{\text{kaug}, i} \right)_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}^{-1}$$

Uses implicit differentiation and KKT conditions to compute $\mathbf{Q}_{\text{kaug}, i}$

$\mathbf{y}$ = measured variables, l = log-likelihood function, n = number of experiments

WSSE = weighted sum of squared errors, $\mathbf{Q}$ = Jacobian, $\boldsymbol{\Sigma}_y$ = measurement error covariance matrix, $\boldsymbol{\theta}$ = parameters, $\mathbf{x}$ = experimental conditions

Notebooks at dowlinglab.github.io/pyomo-doe

Uncertainty Quantification

Alexander Dowling, Dan Laky, Stephen Cini, Shammah Lilonfe, Shuvashish Mondal,
Shilpa Narasimhan

Uncertainty quantification is required to ensure that the estimates of the parameters are close to their true values. This parameter accuracy is measured by computing the covariance matrix. The leading diagonal of this matrix contains the variance of the estimated parameters. The uncertainty in the parameters is taken as the standard deviation of the variance from the covariance matrix.

Assuming Gaussian (correlated or independent and identically distributed) measurement errors, the covariance matrix can be computed using three methods in ParmEst.

Computing the covariance matrix with
ParmEst

Import the necessary packages

Load the experimental data

Get the parameter estimates

Covariance Matrix via Reduced
Hessian

Covariance Matrix via Finite Difference

Covariance Matrix via Automatic
Differentiation

Compare the covariance matrix results

Takeaways from the covariance matrix
results

Parameter estimates (from ParmEst) have high uncertainty

$$U_a = 0.042 \pm 3.3 \times 10^{-5} \text{ W/}^\circ\text{C}, \quad U_b = 0.017 \pm 1.7 \times 10^3 \text{ W/}^\circ\text{C}, \quad (C_{p,H})^{-1} = 0.171 \pm 1.1 \times 10^3 \text{ }^\circ\text{C/J},$$

$$(C_{p,S})^{-1} = 3.138 \pm 3.3 \times 10^5 \text{ }^\circ\text{C/J}$$

Covariance matrix (finite_difference method)

	U_a	U_b	$(C_{p,H})^{-1}$	$(C_{p,S})^{-1}$
U_a	1.1×10^{-9}	5.2×10^{-2}	3.5×10^{-2}	-1.0×10^1
U_b	5.2×10^{-2}	2.9×10^6	1.9×10^6	-5.6×10^8
$(C_{p,H})^{-1}$	3.5×10^{-2}	1.9×10^6	1.3×10^6	-3.8×10^8
$(C_{p,S})^{-1}$	-1.0×10^1	-5.6×10^8	-3.8×10^8	1.1×10^{11}

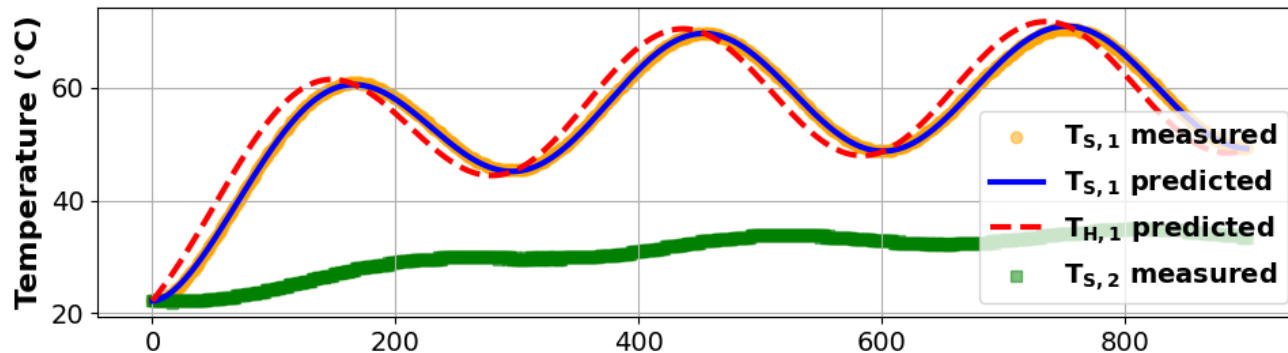
Eigenvalues of V_θ :

$$[1.1 \times 10^{11} \quad 9.2 \times 10^3 \quad 1.4 \times 10^{-10} \quad 1.5 \times 10^3]$$

Eigenvectors of V_θ :

$$\begin{bmatrix} 0.0000 & -0.0000 & 1.0000 & 0.0000 \\ 0.0051 & 0.9929 & 0.0000 & 0.1186 \\ 0.0034 & 0.1186 & 0.0000 & -0.9929 \\ -1.0000 & 0.0055 & 0.0000 & -0.0028 \end{bmatrix} \begin{matrix} U_a \\ U_b \\ (C_{p,H})^{-1} \\ (C_{p,S})^{-1} \end{matrix}$$

Condition number: 6.7×10^{20}



Four ways to possibly improve estimates:

- Multi-start initialization optimization
- Regularization using prior information
- More data collection via experimental design
- Alternative model structures / formulations



Multistart Optimization with ParmEst

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal, Shilpa Narasimhan

Parameter estimation problems are often **nonconvex**, meaning that the objective function $g(\mathbf{x}, \mathbf{y}; \boldsymbol{\theta})$ may contain multiple local minima. As a result, the solution obtained by a nonlinear optimization solver can depend strongly on the initial guess for $\boldsymbol{\theta}$.

Multistart optimization is a practical strategy to address this challenge. Instead of solving the problem once from a single initial point, we solve it multiple times from different initializations and compare the resulting solutions.

CONTENTS ▾

- Import the Necessary Packages
- Load the Experimental Data
- Multistart in ParmEst
 - TC Lab Parameter Estimation Problem Statement
 - Compare Different Multistart Sampling Methods
 - Takeaways from the Multistart Optimization Results

Multistart and Estimability Analysis Further Characterize the Objective Function

Identifiability describes a model or model parameter's ability to be uniquely estimated.

- **Structural** identifiability – **model** focused
- **Practical** identifiability or **estimability** – **data quality** focused

8. Perform multistart optimization

$$\hat{\theta} = \arg \min_{\theta \in \Theta} \frac{1}{2} (\mathbf{y} - \mathbf{f}(\mathbf{x}; \theta))^T \Sigma_y^{-1} (\mathbf{y} - \mathbf{f}(\mathbf{x}; \theta)),$$
$$\Theta = \{ \theta_1, \dots, \theta_p \mid \theta_l \leq \theta_s \leq \theta_u, s = 1, \dots, p \}$$

where Θ is randomly sampled using multiple methods (random uniform, Latin Hypercube Sampling (LHS), or Sobol sampling)

$\mathbf{y}$ = measured variables, L = likelihood function, ϵ = measurement errors, θ = parameters, l = log-likelihood function, n = number of experiments, WSSE = weighted sum of squared errors, $\mathbf{x}$ = experiment conditions, $\mathbf{V}$ = covariance of the measurement error

9. Evaluate estimability with profile likelihood

$$PL_j(\theta_j^*) = \min_{\theta: \theta_j = \theta_j^*} WSSE(\theta)$$
$$2[PL_j(\theta_j^*) - WSSE(\hat{\theta})] \leq \chi_{1,\alpha}^2$$

where one θ , θ_j^* , is fixed and sampled within its bounds, while all the others are estimated, to see how the profile of the objective changes

```
profiles = []
```

```
for pname in profiled_theta_list:  
    for g in grid_values[pname]:
```

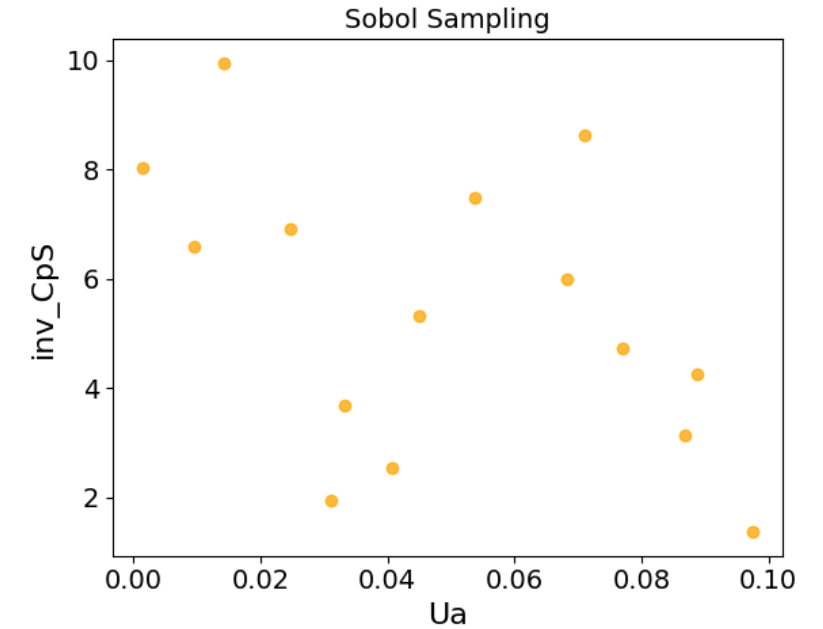
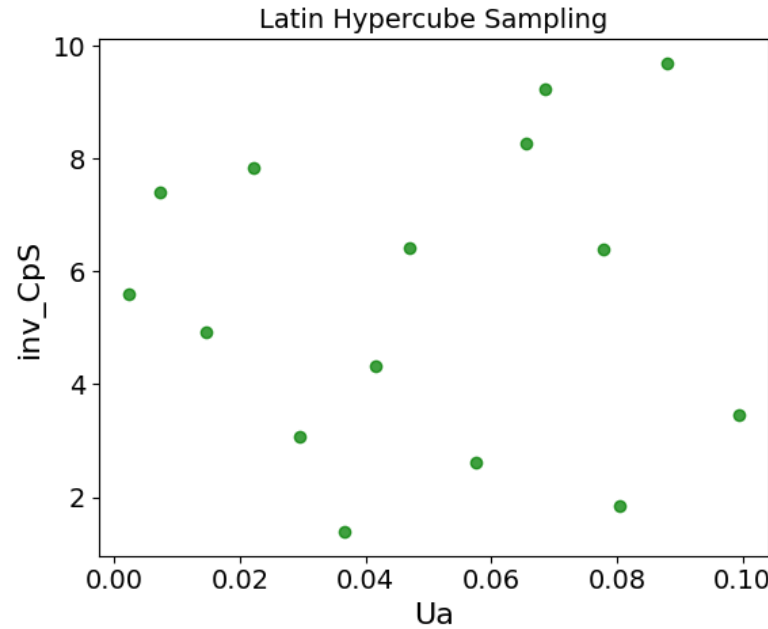
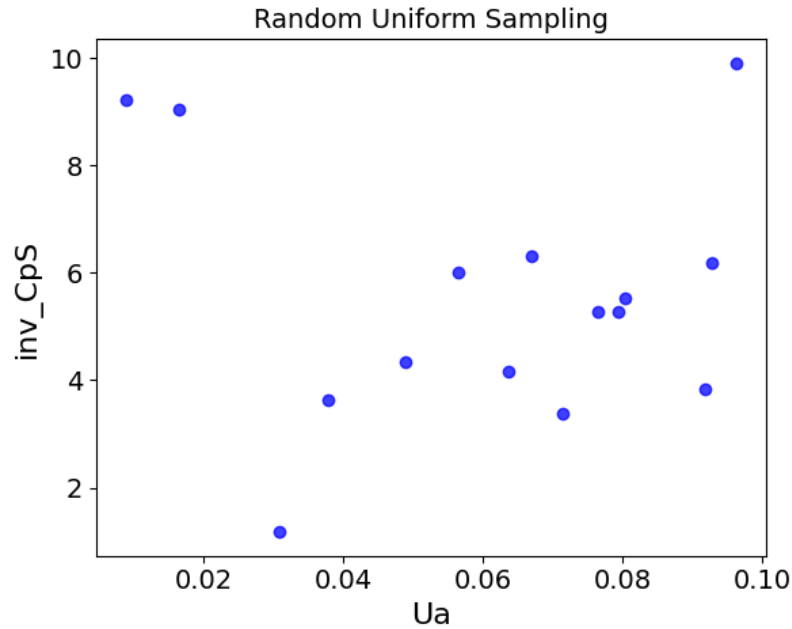
```
        # set the profiled parameter value  
        init_theta[pname] = float(g)
```

```
        # fix pname at g and re-optimize all other parameters  
        obj_val, theta_conv, term = self._Q_opt(  
            theta_vals=init_theta,  
            fixed_theta_values={pname: float(g)},  
        )
```

```
        # compute profile likelihood statistic  
        profile_value = 2.0 * (obj_val - obj_hat)
```

**Profile likelihood
pseudocode**

TCLab: Results from multistart sampling methods



	random_ uniform	latin_ hypercube	sobol_ sampling
Ua	0.041705	0.041705	0.041705
Ub	0.017065	0.017199	0.017104
inv_CpH	0.170701	0.170791	0.170727
inv_CpS	3.137848	3.111786	3.130236
objective	53.773993	53.773993	53.773993

Takeaways:

- Stratified sampling techniques (LHS and Sobol) cover more of sampling space and avoid multiple samples in same region
- Other than Ua, values differ between techniques, but same objective value.
- Similar to with covariance, this points to potential issue with estimability.



Profile Likelihood with ParmEst

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal,
Shilpa Narasimhan

After estimating parameter values, it is important to assess whether those parameters are supported by the available experimental data. In other words, we would like to understand not only which parameter values minimize the objective function, but also how much confidence we should have in those values.

Two related concepts are especially important in parameter estimation:

1. Identifiability

A parameter is identifiable if it can, in principle, be uniquely determined from ideal noise-free data and the specified model structure.

Structural identifiability is a property of the model itself. If a parameter is structurally unidentifiable, then no amount of additional data from the same

CONTENTS ▾

Import the Necessary Packages

Load the Experimental Data

Profile Likelihood for Estimability
Analysis

Interpreting Profile Likelihoods

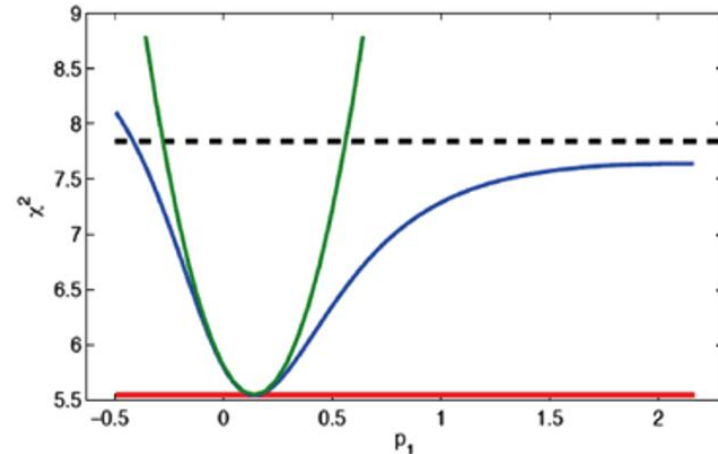
Activity: Discuss Profile Likelihood Results
Above with a Neighbor

Please open the notebook in Colab and run the first cell (takes a few minutes).

Activity: Discuss estimability of parameters based on profile likelihood results

Analyzing profile likelihood:

Identifiable Structural non-identifiable Practical non-identifiable

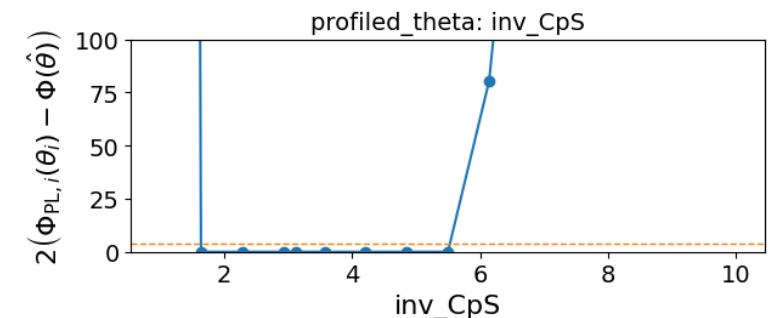
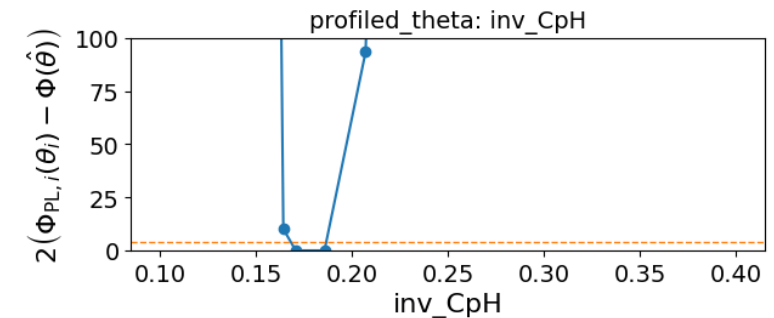
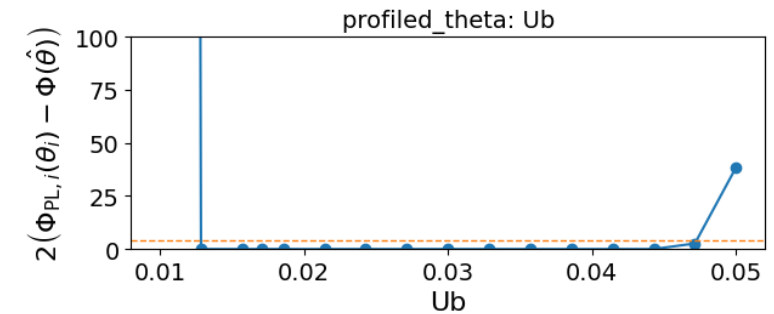
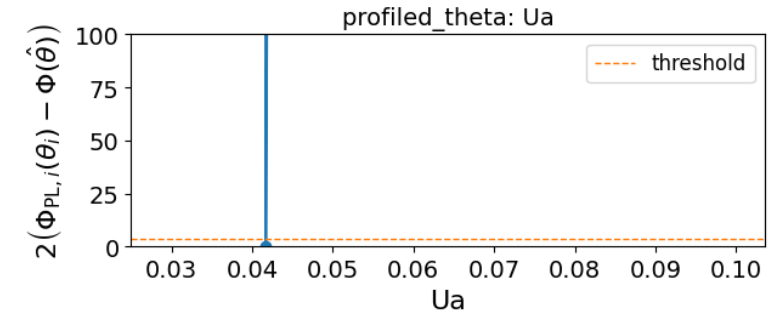


*statistical
significance
threshold*

Bachmann et al (2011), *Journal of Internal Medicine*

From results (used ylims to zoom in):

- U_a is most likely estimable (practically identifiable)
- U_b , CpH^{-1} , and CpS^{-1} are most likely inestimable
- U_b and CpS^{-1} may be structurally nonidentifiable



L2 Regularization with ParmEst

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal,
Shilpa Narasimhan

Parameter estimation problems can be challenging when the available data are limited, noisy, or weakly informative for some parameters. In these cases, the least squares or maximum likelihood objective may have broad valleys, multiple local optima, or parameter estimates that are physically unreasonable.

One way to incorporate additional information is through **regularization**. Regularization augments the parameter estimation objective with an additional term that penalizes deviations from prior parameter values.



CONTENTS ▾

Maximum Likelihood Estimation and Regularization

Import the Necessary Packages

Load Experimental Data

Interpretation of the Prior FIM

Physically Motivated Prior Information

Adding Regularization to Objective in ParmEst

Connection to MAP Estimation

Activity: How Does the Strength of the Prior Affect the Parameter Estimation?

Takeaways from the Activity

Regularization Leverages Prior Parameter Information

Define the Fisher information matrix

$$\mathbf{M}(\boldsymbol{\theta}) = -\mathbb{E} \left(\frac{\partial^2 l(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}} \mid \boldsymbol{\theta} \right)$$

Information contained in experiment design $\mathbf{X}$ and measurements $\mathbf{y}$ about parameters $\boldsymbol{\theta}$ in model $\mathbf{f}(\cdot, \cdot)$

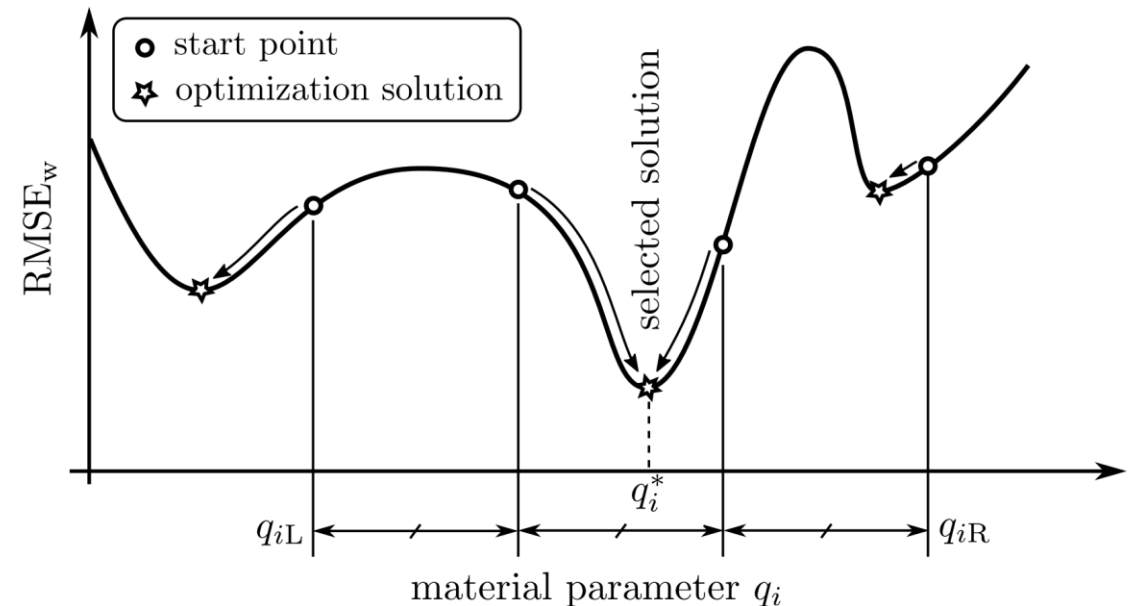
Adding regularization makes the problem equivalent to **Maximum A Posteriori (MAP) estimation**, because we are incorporating more prior information about parameters.

$\mathbf{y}$ = measured variables, L = likelihood function, ϵ = measurement errors, $\boldsymbol{\theta}$ = parameters, l = log-likelihood function, n = number of experiments, WSSE = weighted sum of squared errors, $\mathbf{x}$ = experiment conditions, $\mathbf{V}$ = covariance of the measurement error

Use previous estimate (or physical intuition) $\mathbf{M}$ and $\hat{\boldsymbol{\theta}}$ for regularization

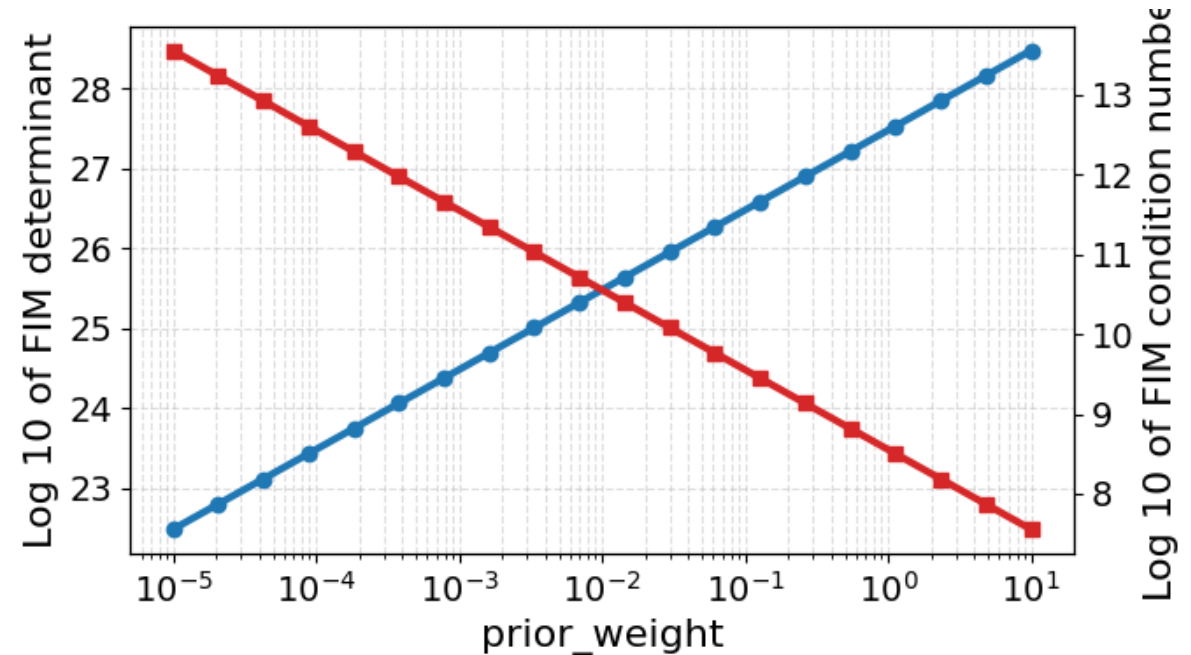
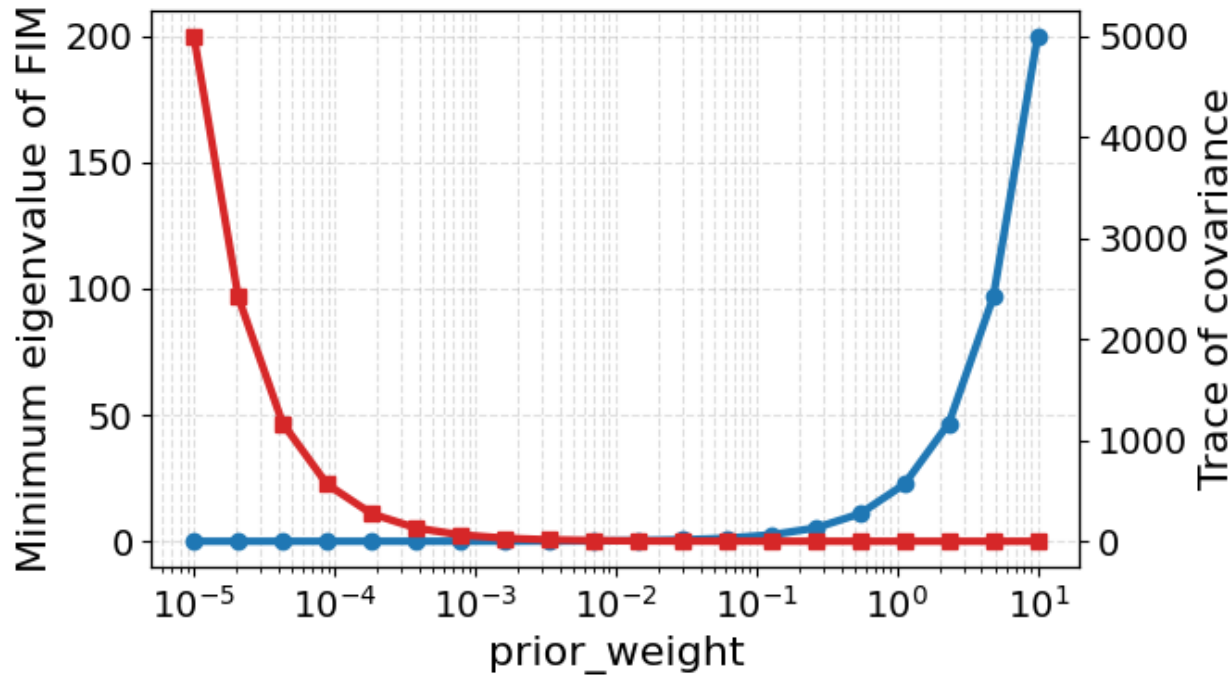
$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \text{WSSE} + \underbrace{\frac{1}{2} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_0)^T \mathbf{M} (\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_0)}_{\text{L2 Regularization (Gaussian Prior)}}$$

L2 Regularization
(Gaussian Prior)



Reisinger et al. *Biomechanics and Modeling in Mechanobiology* (2020)

Activity: How does the strength of the prior affect the parameter estimation?



Takeaways: increasing the regularization weight...

- Raises minimum eigenvalue and determinant of FIM (blue)
- Lowers the FIM condition number and trace of covariance matrix (red)



CONTENTS ▾

Exercise: ParmEst

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal,
Shilpa Narasimhan

In the [main workshop tutorial](#), the four heat transfer parameters, U_a , U_b , C_p^H , C_p^S , were estimated from the sine test data using the following TC Lab model:

$$C_p^H \frac{dT_H}{dt} = U_a(T_{\text{amb}} - T_H) + U_b(T_S - T_H) + \alpha Pu(t), \quad (1)$$

$$C_p^S \frac{dT_S}{dt} = U_b(T_H - T_S) \quad (2)$$

where T_H and T_S are the heater and sensor temperatures in $^{\circ}\text{C}$, respectively, T_{amb} is the ambient temperature in $^{\circ}\text{C}$, C_p^H and C_p^S are the heat capacity of the heater and sensor in $\text{J}/^{\circ}\text{C}$, respectively, U_a and U_b are the heat transfer coefficients from the heater to the sensor and the sensor to ambient (in $\text{W}/^{\circ}\text{C}$), respectively, P is the maximum power limit in `bit`, u is the heater power in $\%$, and α is constant that converts the unit of $Pu(t)$ into W .

Implementing Reformulated Model in Pyomo
Import the Necessary Packages for This Exercise
Load and Explore Experimental Data (sine test)
Parameter Estimation with ParmEst
Quantify the Uncertainty in the Parameter Estimates
Multistart Optimization with ParmEst
Profile Likelihood with ParmEst
Add L2 Regularization to Parameter Estimation Objective

Exercise: Linear model reformulation (w.r.t. parameters) can improve estimability

Original TC Lab model

$$\frac{dT_{H,1}}{dt} = \frac{U_a(T_{amb} - T_{H,1}) + U_b(T_{S,1} - T_{H,1}) + \alpha P_1 u_1(t)}{C_{p,H}},$$

$$\frac{dT_{S,1}}{dt} = \frac{U_b(T_{H,1} - T_{S,1})}{C_{p,S}}$$

Reformulated TC Lab model

$$\frac{dT_{H,1}}{dt} = \beta_1(T_{amb} - T_{H,1}) + \beta_2(T_{S,1} - T_{H,1}) + \beta_4 u_1(t),$$

$$\frac{dT_{S,1}}{dt} = \beta_3(T_{H,1} - T_{S,1})$$

$$\beta_1 = \frac{U_a}{C_{p,H}}, \quad \beta_2 = \frac{U_b}{C_{p,H}}, \quad \beta_3 = \frac{U_b}{C_{p,S}}, \quad \beta_4 = \frac{\alpha P_1}{C_{p,H}}$$

Recover “physical” model parameters θ and covariance

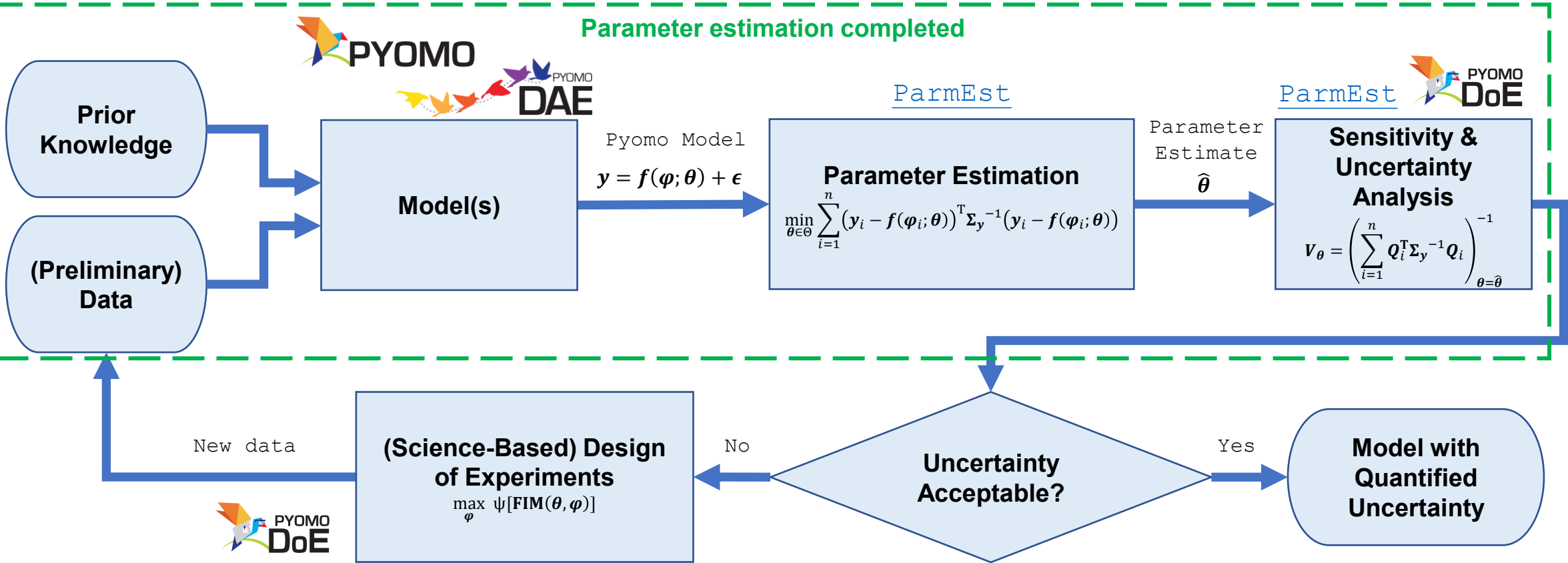
$$U_a = \frac{\alpha P_1 \beta_1}{\beta_4}, \quad U_b = \frac{\alpha P_1 \beta_2}{\beta_4},$$

$$C_{p,H} = \frac{\alpha P_1}{\beta_4}, \quad C_{p,S} = \frac{\alpha P_1 \beta_2}{\beta_3 \beta_4}$$

$$\mathbf{v}_\theta = \left(\frac{\partial \theta}{\partial \beta} \right) \mathbf{v}_\beta \left(\frac{\partial \theta}{\partial \beta} \right)^T$$

U_a = heater heat transfer coefficient in W/°C,
 U_b = sensor heat transfer coefficient in W/°C,
 $C_{p,H}$ = heat capacity of the heater in J/°C, t = time in s,
 $T_{H,1}$ = temperature of heater 1 in °C,
 $T_{S,1}$ = temperature of sensor 1 in °C,
 T_{amb} = ambient temperature in °C,
 $C_{p,S}$ = heat capacity of the sensor in J/°C,
 u_1 = heater 1 power in %,
 P_1 = hardware-set maximum power limit in bit,
 α = known constant for converting the unit of $P_1 u_1$ into W

Estimating the four parameters follows an iterative approach



Pyomo is an open-source Python package for optimization modeling, **ParmEst** is a Pyomo tool for parameter estimation, **Pyomo.DoE** is a Pyomo tool for design of experiments, θ = unknown parameters, y = observation of measured variables, **FIM** = Fisher information matrix, V_{θ} = parameter covariance matrix, φ = design vector, ψ = scalar measure (e.g., determinant), Q = Jacobian matrix, ϵ = measurement error, Σ_y = measurement error covariance matrix, and n = number of experiments

Agenda

Part 1 (12:15pm – 1:30pm): Welcome to Pyomo

- TCLab Mathematical Model
- Simulation and Dynamic Optimization in Pyomo
- Experiment Abstraction

Part 2 (1:45pm – 3pm): Parameter Estimation

- ParmEst Overview
- Uncertainty Quantification
- Multistart & Profile Likelihood
- Regularization

Part 3 (3:30pm – 5pm): Optimal Experiment Design

- Pyomo.DoE Overview
- OED Objectives
- Symbolic Derivatives
- Planning Multiple Experiments





CONTENTS ▾

Getting Started with Pyomo.DoE

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal, Shilpa Narasimhan

Model-based design uses physics-based models. It starts with prior knowledge and iterates through the loop until a satisfactory model is found. We have already discussed parameter estimation in the first part of the workshop. If we have multiple models, we can discriminate among them to find the best model that fits the data. However, that is not a part of this workshop.

*Please open the notebook in Colab
and run the first cell (takes a few
minutes).*

Fisher Information Matrix

Scaling in Pyomo.DoE

Optimization Formulation

Optimal Experiment Design

Import Necessary Functions

Load Experimental Data (sine test)

Use Prior Parameter Information

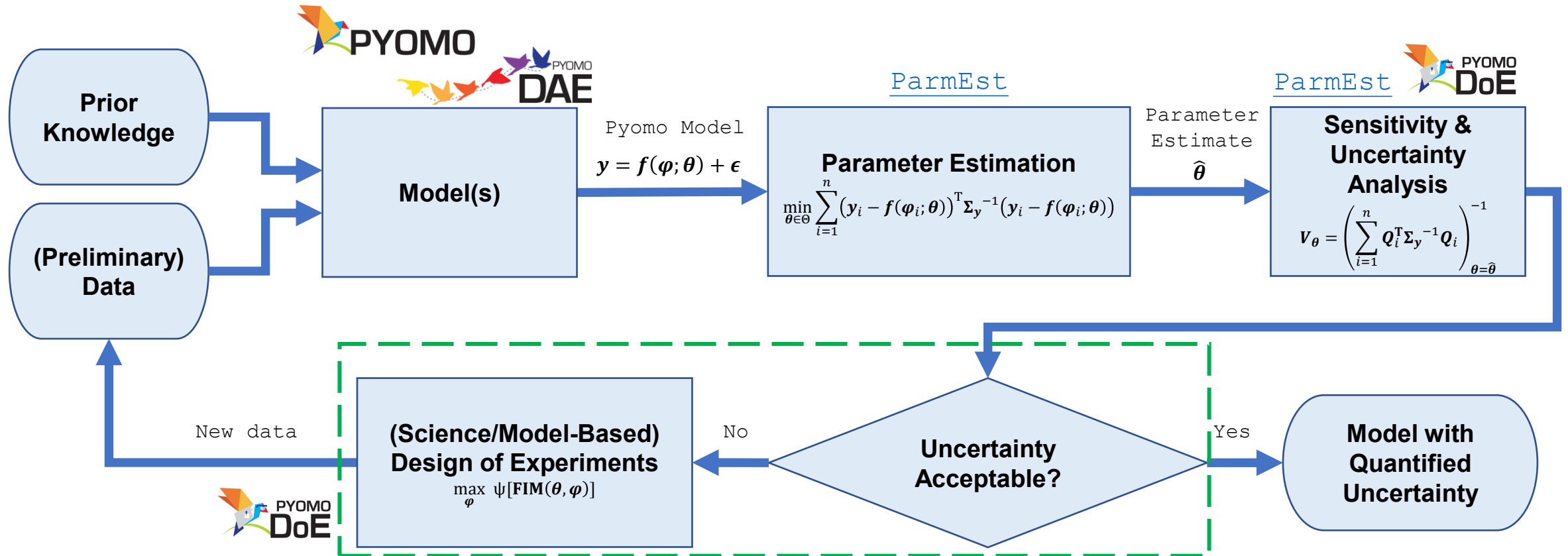
Optimize Next Experiment (Pseudo-A-Optimality)

Compare the Reduction in Predicted Uncertainty

Optimize Experiment with New Initial Point

Analyze the New Result

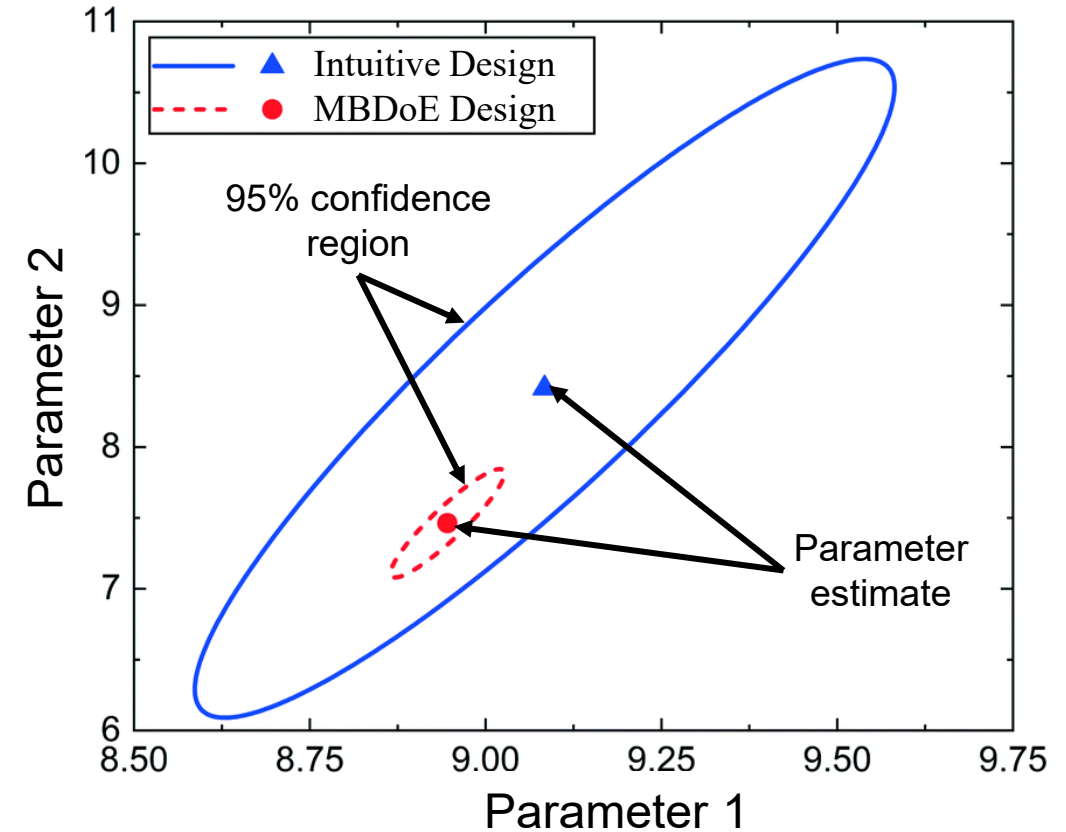
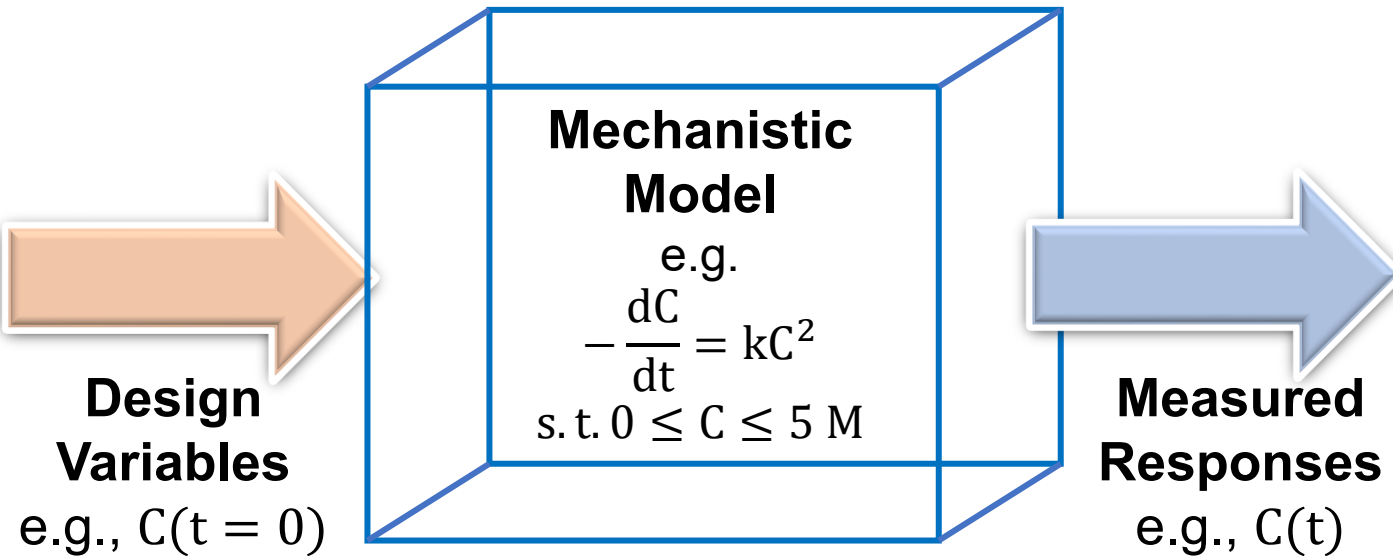
Optimizing new experiments



Pyomo is an open-source Python package for optimization modeling, **ParmEst** is a Pyomo tool for parameter estimation, **Pyomo.DoE** is a Pyomo tool for design of experiments, θ = unknown parameters, y = observation of measured variables, **FIM** = Fisher information matrix, V_{θ} = parameter covariance matrix, φ = design vector, ψ = scalar measure (e.g., determinant), Q = Jacobian matrix, ϵ = measurement error, Σ_y = measurement error covariance matrix, and n = number of experiments

Model-based design of experiments (MBDoe) optimizes information gained from next experiment(s)

MBDoE



Waldron et al. (2020), *Reaction Chemistry & Engineering*

MBDoE uses a mechanistic model to optimize experimental conditions that improve parameter precision¹

MBDoE can reduce parameter uncertainty relative to intuitive designs

Model-Based DoE Optimization Formulation

$$\begin{aligned}
 & \max_{\varphi} \quad \Psi[\mathbf{M}(\hat{\boldsymbol{\theta}}, \boldsymbol{\varphi})] \\
 & \text{s. t.} \quad \left. \begin{aligned}
 & \dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{z}(t), \mathbf{u}(t), \bar{\mathbf{w}}, \hat{\boldsymbol{\theta}}) \\
 & \mathbf{g}(\mathbf{x}(t), \mathbf{z}(t), \mathbf{u}(t), \bar{\mathbf{w}}, \hat{\boldsymbol{\theta}}) = \mathbf{0} \\
 & \mathbf{y}(t) = \mathbf{h}(\mathbf{x}(t), \mathbf{z}(t), \hat{\boldsymbol{\theta}})
 \end{aligned} \right\} \text{DAE System} \\
 & \quad \quad \quad \left. \begin{aligned}
 & \mathbf{f}^0(\dot{\mathbf{x}}(t_0), \mathbf{x}(t_0), \mathbf{z}(t_0), \mathbf{u}(t_0), \bar{\mathbf{w}}, \hat{\boldsymbol{\theta}}) = \mathbf{0} \\
 & \mathbf{g}^0(\mathbf{x}(t_0), \mathbf{z}(t_0), \mathbf{u}(t_0), \bar{\mathbf{w}}, \hat{\boldsymbol{\theta}}) = \mathbf{0} \\
 & \mathbf{y}^0(t_0) = \mathbf{h}(\mathbf{x}(t_0), \mathbf{z}(t_0), \hat{\boldsymbol{\theta}})
 \end{aligned} \right\} \text{Initial Conditions}
 \end{aligned}$$

We will start with **pseudo-A optimality**:
 $\Psi(\mathbf{M}) = \log_{10}(\text{trace}(\mathbf{M}))$

$\mathbf{m}(\mathbf{x}(t), \mathbf{y}(t), \mathbf{z}(t), \mathbf{u}(t), \bar{\mathbf{w}}, \hat{\boldsymbol{\theta}}) = \mathbf{0}$

- $\mathbf{y}$ Measurements (model responses)
- $\hat{\boldsymbol{\theta}}$ Estimated parameters
- $\mathbf{x}$ Time-dependent differential state variables
- $\mathbf{z}$ Time-dependent algebraic state variables
- $\mathbf{u}$ Time-varying control variables
- $\bar{\mathbf{w}}$ Time-invariant control variable

Fisher information matrix (FIM):

$$\mathbf{M} \approx \mathbf{V}_{\hat{\boldsymbol{\theta}}}^{-1} \approx \sigma_{\epsilon}^{-2} \mathbf{H} \approx \sigma_{\epsilon}^{-2} \mathbf{Q}^T \mathbf{Q}$$

MBDoE Decisions:

$$\boldsymbol{\varphi} = (\mathbf{u}(t), \mathbf{x}(t_0), \mathbf{z}(t_0), \bar{\mathbf{w}}, \mathbf{t})$$

Scaling in Pyomo.DoE

1. Constant scaling (`scale_constant_value`)

Multiplies every element of the sensitivity matrix $\mathbf{Q} = \left. \frac{\partial \hat{\mathbf{y}}}{\partial \boldsymbol{\theta}} \right|_{\hat{\boldsymbol{\theta}}}$ by a single uniform scalar.

Impact: Shifts the overall magnitude to avoid floating-point errors but **does not improve the matrix condition number**.

2. Nominal Parameter Scaling (`scale_nominal_param_value`)

Scales each column of the sensitivity matrix by its corresponding nominal parameter value.

Impact: **Directly improves the condition number** of the sensitivity matrix and the resulting FIM. This is the preferred choice when parameter ranges span multiple orders of magnitude.

$$\mathbf{Q}_{\text{scaled}} = \begin{bmatrix} \frac{\partial y_1}{\partial \theta_1} \theta_1 & \frac{\partial y_1}{\partial \theta_2} \theta_2 & \cdots & \frac{\partial y_1}{\partial \theta_p} \theta_p \\ \frac{\partial y_2}{\partial \theta_1} \theta_1 & \frac{\partial y_2}{\partial \theta_2} \theta_2 & \cdots & \frac{\partial y_2}{\partial \theta_p} \theta_p \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial \theta_1} \theta_1 & \frac{\partial y_n}{\partial \theta_2} \theta_2 & \cdots & \frac{\partial y_n}{\partial \theta_p} \theta_p \end{bmatrix}$$

If we use scaling, then we also need to ensure that the prior FIM is also scaled accordingly and after designing the experiments, we also need to unscale the FIM to make a fair comparison

TC Lab: Pseudo-A-Optimal Next Experiment

dowlinglab.github.io/pyomo-doe/notebooks/doe_optimize.html

$$\max_u \log \text{trace}(\mathbf{M}(u) + \mathbf{M}_0)$$

$$\text{s.t. } C_p^H \frac{dT_H}{dt} = \dots$$

$$C_p^S \frac{dT_S}{dt} = \dots$$

$$0\% \leq u(t) \leq 100\%$$

$$T_H(t_0) = T_{amb}$$

$$T_S(t_0) = T_{amb}$$

Parameters $\pm$ SD (Prior)

Ua: $4.170\text{e-}02 \pm 1.363\text{e-}05$

Ub: $1.201\text{e-}02 \pm 4.030\text{e-}04$

inv_CpH: $1.675\text{e-}01 \pm 3.212\text{e-}04$

inv_CpS: $4.545\text{e+}00 \pm 1.581\text{e-}01$

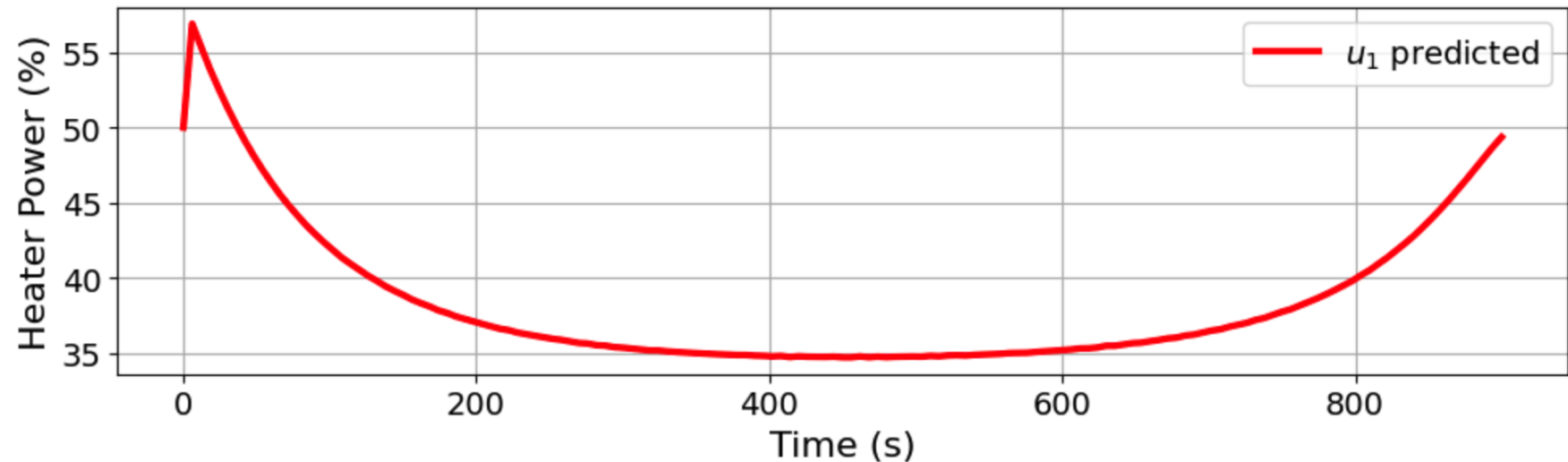
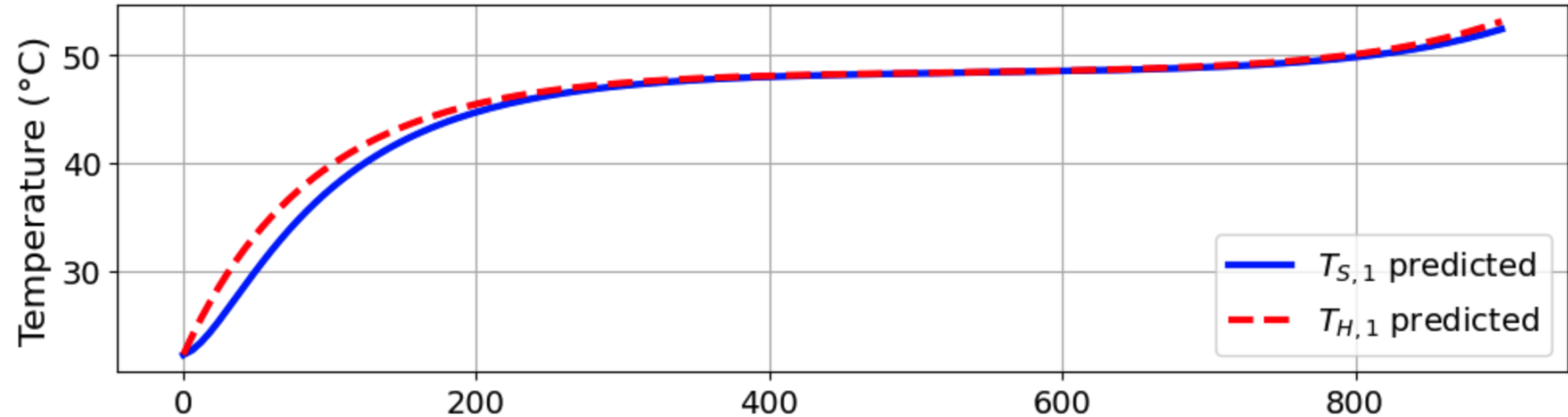
Parameters $\pm$ SD (Design)

Ua: $4.170\text{e-}02 \pm 1.363\text{e-}05$

Ub: $1.201\text{e-}02 \pm 4.877\text{e-}05$

inv_CpH: $1.675\text{e-}01 \pm 2.064\text{e-}04$

inv_CpS: $4.545\text{e+}00 \pm 6.429\text{e-}04$



pseudo-A optimal design improves the estimability of C_p^S



CONTENTS ▾

Experiment Design Objectives in Pyomo.DoE

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal, Shilpa Narasimhan

In the [previous notebook](#), we formulated model-based design of experiments as an optimal control problem using one Fisher information matrix (FIM)-based objective.

In this workbook, we extend that idea to additional optimality criteria. We will use Pyomo.DoE to formulate, initialize, and solve experiment design problems using D-, A-, pseudo A-, E-, and ME-optimality objectives. This lets us compare how the choice of design criterion changes the optimized input profile, $u(t)$, and the information content of the resulting experiment.

- Objective Options in Pyomo.DoE
- Load Experimental Data (sine test)
- Optimize Next Experiment (A-Optimality)
- E-Optimality using GreyBox Formulation
- Sensitivity Analysis
- Modified E (ME)-Optimality
- Optimize Next Experiment (D-Optimality)
- FIM Comparison for All Optimality

Please open the notebook in Colab and run the first cell (takes a few minutes).

Alphabetic Design Criteria Measure Information Content

Figure adapted from: Franceschini, G., & Macchietto, S. (2008). *Chem. Eng. Sci.*, 63(19), 4846-4872.

A-optimality

$$\min \text{trace}(\mathbf{V}_{\theta})$$

enclosing box volume

poor choice for highly correlated θ

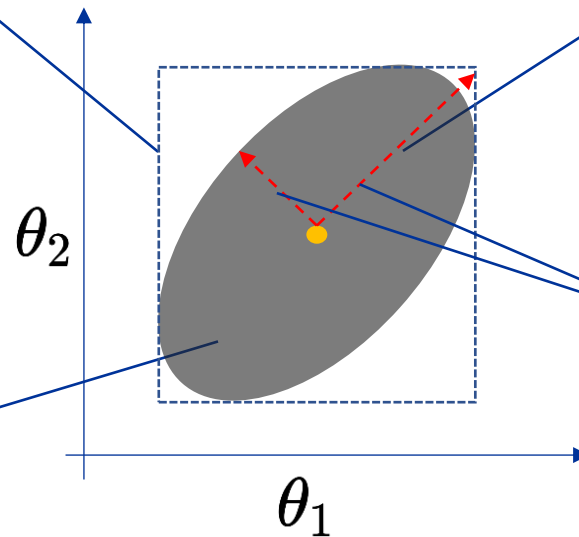
D-optimality

$$\max \det(\mathbf{M}) = 1 / \min \det(\mathbf{V}_{\theta})$$

ellipsoid volume

robust to linear transformations

confidence ellipsoid for
covariance matrix $\mathbf{V}_{\theta} \approx \mathbf{M}^{-1}$



E-optimality

$$\max \min(\text{eig}(\mathbf{M})) = \min \max \text{eig}(\mathbf{V}_{\theta})$$

major axis

recommended if $\mathbf{M}$ is ill-conditioned

ME-optimality

$$\min \kappa(\mathbf{M}) \stackrel{\text{def}}{=} \max(\text{eig}(\mathbf{M})) / \min(\text{eig}(\mathbf{M}))$$

ratio of major to minor axes

recommended if $\mathbf{M}$ is ill-conditioned

Model Discrimination

Hunter, W.G. and Reiner, A.M., 1965. Designs for discriminating between two rival models. *Technometrics*, 7(3), pp.307-323.

Buzzi-Ferraris, G. and Forzatti, P., 1983. A new sequential experimental design procedure for discriminating among rival models. *Chemical engineering science*, 38(2), pp.225-232.

Ferraris, G.B., Forzatti, P., Emig, G. and Hofmann, H., 1984. Sequential experimental design for model discrimination in the case of multiple responses. *Chemical engineering science*, 39(1), pp.81-85.

Joint Parameter Precision and Model Discrimination

Alberton, A.L., Schwaab, M., Lobão, M.W.N. and Pinto, J.C., 2011. Experimental design for the joint model discrimination and precise parameter estimation through information measures. *Chemical Engineering Science*, 66(9), pp.1940-1952.

Galvanin, F., Cao, E., Al-Rifai, N., Gavrilidis, A. and Dua, V., 2016. A joint model-based experimental design approach for the identification of kinetic models in continuous flow laboratory reactors. *Computers & Chemical Engineering*, 95, pp.202-215.

Galvanin, F., Cao, E., Al-Rifai, N., Dua, V. and Gavrilidis, A., 2015. Optimal design of experiments for the identification of kinetic models of methanol oxidation over silver catalyst. *Chimica Oggi-Chemistry Today*, 33(3), pp.51-56.

Pankajakshan, A., Waldron, C., Quaglio, M., Gavrilidis, A. and Galvanin, F., 2019. A Multi-Objective Optimal Experimental Design Framework for Enhancing the Efficiency of Online Model Identification Platforms. *Engineering*, 5(6), pp.1049-1059.

Default Pyomo.DoE Formulation: MBDoE as 2-Stage Program

$$\begin{aligned}
 &\max \quad \log \det(\mathbf{M}(\hat{\boldsymbol{\theta}}, \boldsymbol{\varphi})) = 2 \sum_{i=1}^{N_p} \log L_{ii} \quad \text{D-optimality} \\
 &\text{s.t.} \quad \mathbf{M} = \sum_r \sum_{r'} \tilde{\sigma}_{r,r'} \mathbf{Q}_r^T \mathbf{Q}_{r'} \quad \text{Stage 1} \\
 &\quad \mathbf{M} = \mathbf{L}\mathbf{L}^T, \quad L_{ii} \geq \epsilon \quad \text{Cholesky factorization} \\
 &\quad q_{r,p}(t) = \frac{y_{r,p}^+(t) - y_{r,p}^-(t)}{2 \epsilon_p} \quad \text{Central finite difference} \\
 &\quad \mathbf{m}(\mathbf{x}_p^+(t), \mathbf{y}_p^+(t), \mathbf{z}_p^+(t), \mathbf{u}(t), \bar{\mathbf{w}}, \boldsymbol{\theta}_p^+) = \mathbf{0} \quad \text{Two model evaluations} \\
 &\quad \mathbf{m}(\mathbf{x}_p^-(t), \mathbf{y}_p^-(t), \mathbf{z}_p^-(t), \mathbf{u}(t), \bar{\mathbf{w}}, \boldsymbol{\theta}_p^-) = \mathbf{0} \\
 &\quad \boldsymbol{\theta}_p^+ = \hat{\boldsymbol{\theta}} + \mathbf{e}_p \epsilon_p \quad \text{Up and down perturbations} \\
 &\quad \boldsymbol{\theta}_p^- = \hat{\boldsymbol{\theta}} - \mathbf{e}_p \epsilon_p \\
 &\quad \text{Stage 2} \\
 &\quad \forall p \in \{1, \dots, N_p\}
 \end{aligned}$$

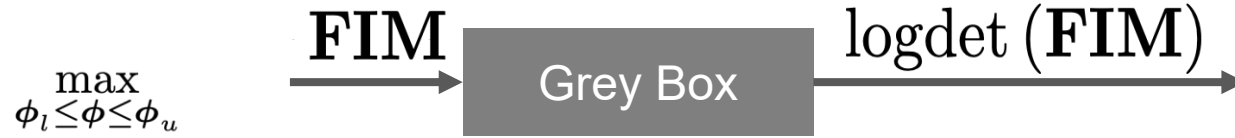
Model Sensitivity

$$\mathbf{Q}_r = \begin{bmatrix} \frac{\partial y_r(t_1)}{\partial \theta_1} & \cdots & \frac{\partial y_r(t_1)}{\partial \theta_{N_p}} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_r(t_n)}{\partial \theta_1} & \cdots & \frac{\partial y_r(t_n)}{\partial \theta_{N_p}} \end{bmatrix} = [\mathbf{q}_{r,1} \quad \cdots \quad \mathbf{q}_{r,N_p}]$$

$$\mathbf{q}_{r,p} = \left[\frac{\partial y_r(t_1)}{\partial \theta_p} \quad \cdots \quad \frac{\partial y_r(t_n)}{\partial \theta_p} \right]^T$$

$\mathbf{y}$	Measurements (model responses)
$\mathbf{Q}_r$	Dynamic sensitivity for response r
$\mathbf{m}(\cdot)$	DAE model
$\hat{\boldsymbol{\theta}} \in \mathbb{R}^P$	Estimate for parameters
$\mathbf{M} \in \mathbb{R}^{P \times P}$	Fisher information matrix
$\mathbf{L} \in \mathbb{R}^{P \times P}$	Lower triangular Cholesky factorization
ϵ_p	Small perturbation for parameter p
$\mathbf{e}_p \in \mathbb{R}^P$	Unit vector with “1” in position p

Improving Computational Performance by Simplifying the Optimal DoE Problem Formulation



Callback to a “grey box” function allows D-optimality calculation without Cholesky Factorization

$$\max_{\phi_l \leq \phi \leq \phi_u}$$

s.t.

$$\mathbf{FIM}(\hat{\theta}, \phi) = \sum_r^{N_r} \sum_{r'}^{N_r} \tilde{\sigma}_{(r,r')} \mathbf{Q}_r^T \mathbf{Q}_{r'} + \mathbf{FIM}_{\text{prior}}$$

$$\mathbf{q}_{r,p} = \frac{\mathbf{y}_{r,p}^+ - \mathbf{y}_{r,p}^-}{2\epsilon_p} \quad \forall r \in \{1, \dots, N_{\text{meas}}\} \forall p \in \{1, \dots, N_p\}$$

$$\mathbf{y}_{r,p}^+ = m(x_p, \boldsymbol{\theta}_p^+) \quad \forall p \in \{1, \dots, N_p\}$$

$$\mathbf{y}_{r,p}^- = m(x_p, \boldsymbol{\theta}_p^-) \quad \forall p \in \{1, \dots, N_p\}$$

$$\boldsymbol{\theta}_p^+ = \hat{\boldsymbol{\theta}} + \mathbf{e}_p \epsilon_p \quad \forall p \in \{1, \dots, N_p\}$$

$$\boldsymbol{\theta}_p^- = \hat{\boldsymbol{\theta}} - \mathbf{e}_p \epsilon_p \quad \forall p \in \{1, \dots, N_p\}$$

This requires derivatives of the outputs: $\log\det(\mathbf{FIM})$ with respect to the inputs: $\mathbf{FIM}$

Jacobian
$$\frac{\partial \log\det(\mathbf{FIM})}{\partial \mathbf{FIM}} = \frac{1}{2} (\mathbf{FIM}^{-1} + \mathbf{FIM}^{-T})$$

Hessian
$$\frac{\partial}{\partial \mathbf{FIM}_{kl}} \left(\frac{\partial \log\det(\mathbf{FIM})}{\partial \mathbf{FIM}} \right)_{ij} = (\mathbf{FIM}^{-1})_{il} (\mathbf{FIM}^{-1})_{kj}$$

Some Common Criteria Cannot Be Generally Posed in Equation-Oriented Formulations



Dan Laky

Laky et al (2026), Optimal Experimental Design using Eigenvalue-Based Criteria with Pyomo.DoE, *arXiv:2604.03354*

$$\max_{\phi_l \leq \phi \leq \phi_u} \Psi \left(\text{FIM} \left(\hat{\theta}, \phi \right) \right)$$

$$\text{s.t.} \quad \text{FIM} \left(\hat{\theta}, \phi \right) = \sum_r \sum_{r'} \tilde{\sigma}_{(r,r')} \mathbf{Q}_r^T \mathbf{Q}_{r'} + \text{FIM}_{\text{prior}}$$

$$\mathbf{q}_{r,p} = \frac{\mathbf{y}_{r,p}^+ - \mathbf{y}_{r,p}^-}{2\epsilon_p} \quad \forall r \in \{1, \dots, N_{\text{meas}}\} \quad \forall p \in \{1, \dots, N_p\}$$

$$\mathbf{y}_{r,p}^+ = m \left(x_p, \boldsymbol{\theta}_p^+ \right) \quad \forall p \in \{1, \dots, N_p\}$$

$$\mathbf{y}_{r,p}^- = m \left(x_p, \boldsymbol{\theta}_p^- \right) \quad \forall p \in \{1, \dots, N_p\}$$

$$\boldsymbol{\theta}_p^+ = \hat{\boldsymbol{\theta}} + \mathbf{e}_p \epsilon_p \quad \forall p \in \{1, \dots, N_p\}$$

$$\boldsymbol{\theta}_p^- = \hat{\boldsymbol{\theta}} - \mathbf{e}_p \epsilon_p \quad \forall p \in \{1, \dots, N_p\}$$



Eigenvalue-based metrics cannot be algebraically expressed beyond 4 parameters

E-optimality

$\max \min(\text{eig}(\mathbf{M}))$

major axis

recommended if $\mathbf{M}$ is ill-conditioned

ME-optimality

$\min \kappa(\mathbf{M}) = \max(\text{eig}(\mathbf{M})) / \min(\text{eig}(\mathbf{M}))$

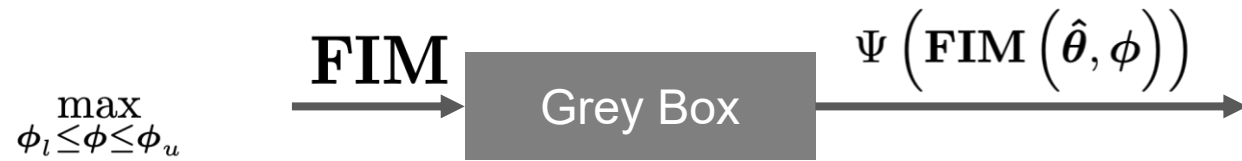
ratio of major to minor axes

recommended if $\mathbf{M}$ is ill-conditioned

GreyBox to Evaluate Eigenvalue Metrics



Dan Laky



s.t.

$$\text{FIM}(\hat{\theta}, \phi) = \sum_r^{N_r} \sum_{r'}^{N_r} \tilde{\sigma}_{(r,r')} \mathbf{Q}_r^T \mathbf{Q}_{r'} + \text{FIM}_{\text{prior}}$$

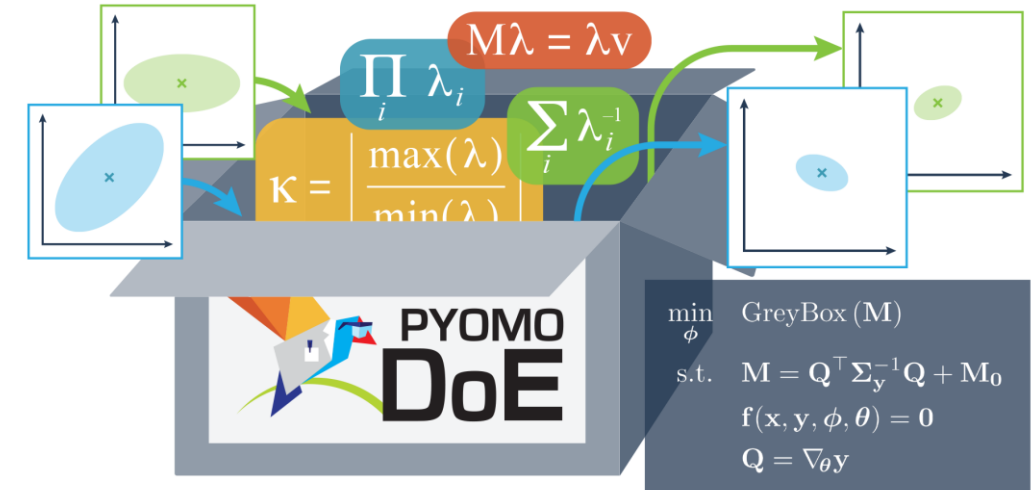
$$\mathbf{q}_{r,p} = \frac{\mathbf{y}_{r,p}^+ - \mathbf{y}_{r,p}^-}{2\epsilon_p} \quad \forall r \in \{1, \dots, N_{\text{meas}}\} \quad \forall p \in \{1, \dots, N_p\}$$

$$\mathbf{y}_{r,p}^+ = m(x_p, \boldsymbol{\theta}_p^+) \quad \forall p \in \{1, \dots, N_p\}$$

$$\mathbf{y}_{r,p}^- = m(x_p, \boldsymbol{\theta}_p^-) \quad \forall p \in \{1, \dots, N_p\}$$

$$\boldsymbol{\theta}_p^+ = \hat{\boldsymbol{\theta}} + \mathbf{e}_p \epsilon_p \quad \forall p \in \{1, \dots, N_p\}$$

$$\boldsymbol{\theta}_p^- = \hat{\boldsymbol{\theta}} - \mathbf{e}_p \epsilon_p \quad \forall p \in \{1, \dots, N_p\}$$

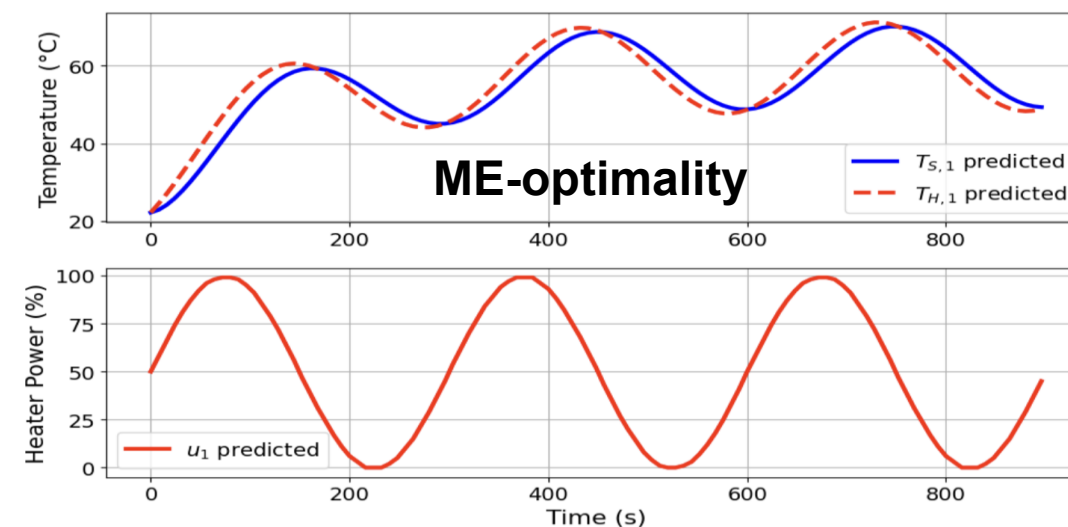
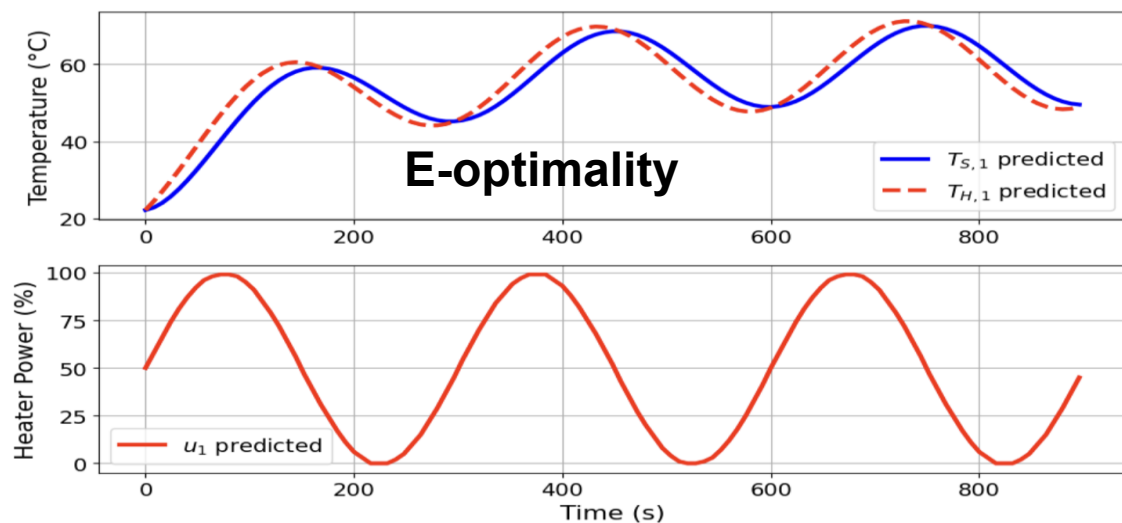
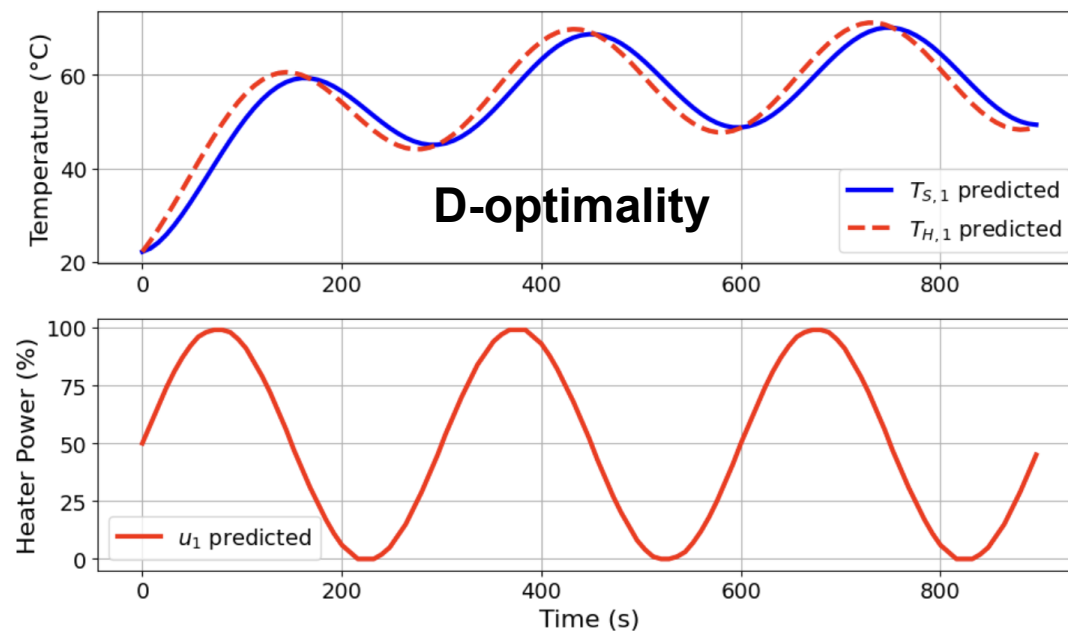
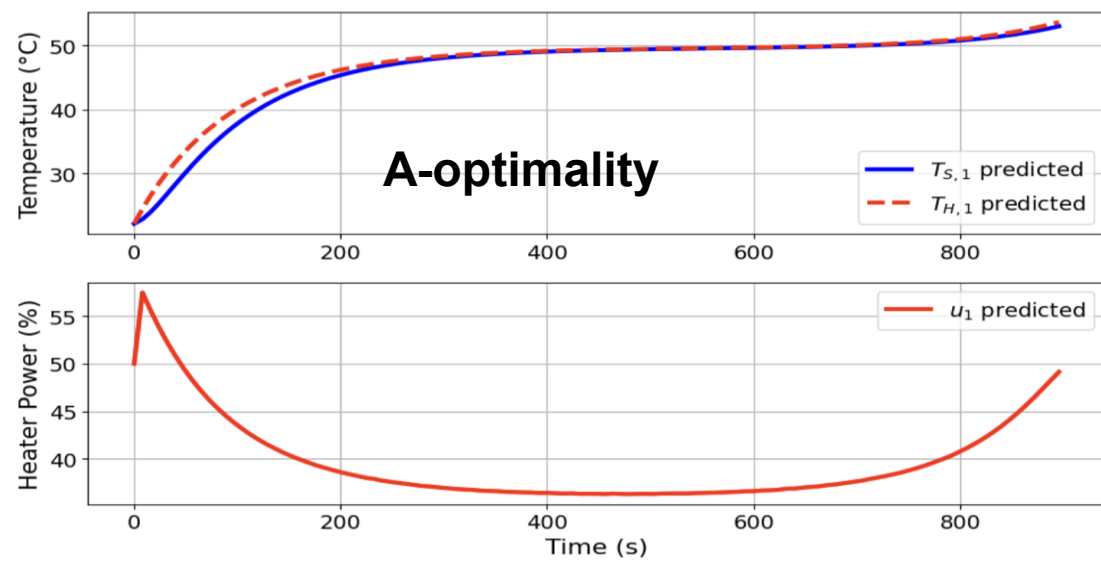


arXiv:2604.03354

Key Contribution: Derivation of exact first and second derivative formulas of metrics Ψ with respect to elements of $\mathbf{M}$

Take Away: Eigenvalue objectives now available for many unknown parameters within an equation-oriented optimization framework (Pyomo)

Optimized Experiments from Different Design Criteria



Cross-Evaluation of Optimized Experiment Designs

Experiment Design	$\log_{10}(M(u) + M_0)^{-1}$	$\log_{10}\det(M(u) + M_0)$	$\log_{10}\lambda_{\min}(M(u) + M_0)$	$\log_{10}\kappa(M(u) + M_0)$	Design decision
Prior FIM	-1.6	27.78	1.6	8.25	N/A
A-optimal	-7.34	36.12	7.36	2.59	Maximize overall information
D-optimal	-4.66	30.84	4.66	5.20	Maximize average uncertainty
E-optimal	-4.66	30.84	4.66	5.20	Improve weak direction
ME-optimal	-4.72	30.90	4.72	5.14	Better conditioned than E

ME reduces eigenvalue spread relative to E-opt



CONTENTS ▾

Computing Sensitivity Matrix in Pyomo.DoE

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal, Shilpa Narasimhan

Recall that `Pyomo.DoE` utilizes the sensitivity matrix defined as the derivative of the estimates of the measured outputs $\hat{y}$ with respect to the parameter to be estimated $\hat{\theta}$:

$$Q = \nabla_{\hat{\theta}} \hat{y} \quad (1)$$

to compute the FIM as:

$$M = Q^T \Sigma_y^{-1} Q \quad (2)$$

where Σ_y is the measurement covariance matrix

Load Experimental Data (sine test)
Calculate FIM at Initial Point (Values from L_2 Regularization)
Optimize Next Experiment (E-Optimality) using Central Finite Difference Method
Run E-Optimality Experiment using Automatic Differentiation
Comparison of the Central Finite Difference and Automatic Differentiation Methods' Computational Time

Please open the notebook in Colab and run the first cell (takes a few minutes).

Sensitivity Computation Using Symbolic Derivatives

Sensitivity matrix $Q = \nabla_{\hat{\theta}} \hat{y}$ feeds the FIM: $M = Q^T \Sigma_y^{-1} Q$ where Σ_y is the measurement covariance matrix

1. Central Finite Difference

$$Q \approx \frac{\hat{y}(\hat{\theta} + \Delta\hat{\theta}) - \hat{y}(\hat{\theta} - \Delta\hat{\theta})}{2\Delta\hat{\theta}}$$

where $\Delta\hat{\theta}$ is the perturbation step size

Requires $2n_p + 1$ copies of the base model to solve

Optimization problem size significantly **increases with number of parameters**

Default/legacy approach in Pyomo.DoE

2. Automatic Differentiation (PyNumero)

Implicit model of the system:

$$F(\hat{x}, \hat{y}, \hat{\theta}, u) = 0$$

PyNumero computes derivatives of F : $\nabla_{\hat{x}} F$, $\nabla_{\hat{y}} F$, $\nabla_{\hat{\theta}} F$.
These are rearranged to obtain:

$$\nabla_{\hat{\theta}} F = \begin{bmatrix} \nabla_{\hat{y}} F \\ \nabla_{\hat{x}} F \end{bmatrix} \begin{bmatrix} \nabla_{\hat{\theta}} \hat{y} \\ \nabla_{\hat{\theta}} \hat{x} \end{bmatrix} = Q$$

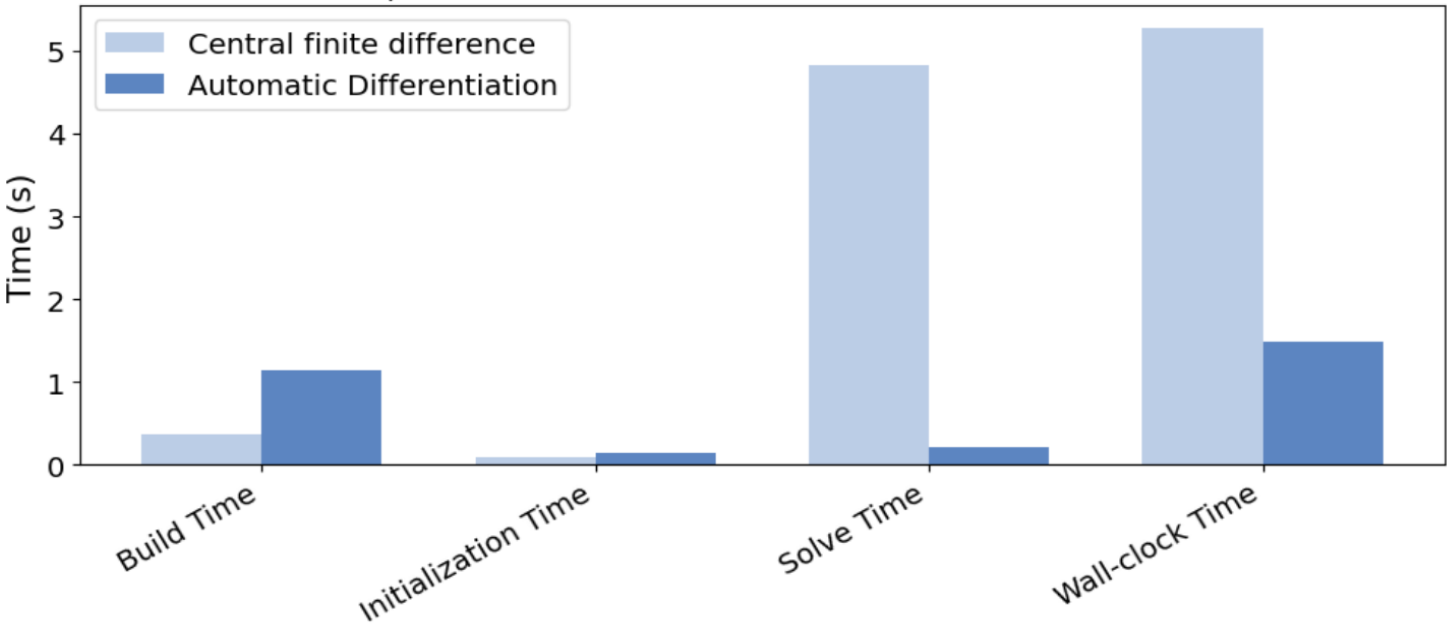
The output sensitivity needed for the FIM is: $Q = \nabla_{\hat{\theta}} \hat{y}$

No repeated parameter-perturbation solves are required.

New feature in Pyomo.DoE

Sensitivity computation within Pyomo.DoE: A Comparison

Time comparison: Central finite difference vs Automatic differentiation



Key Takeaways

- Symbolic (AD) build time longer due to derivative setup
- Symbolic (AD) greatly reduces solve times by avoiding repeated parameter perturbations
- Overall speedup: approximately 3.5 times faster based on wall-clock time
- Symbolic (AD) is likely beneficial for large problems with many parameters

Time Component	Finite Difference	Symbolic (Automatic Differentiation (AD))
Build Time	0.4 s	1.2 s
Solve Time	0.1 s	0.15 s
Initialization Time	4.8 s	0.2 s
Wall-clock Time	5.3 s	1.5 s



CONTENTS ▾

Planning Multiple Experiments with Pyomo.DoE

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal, Shilpa Narasimhan

Multi-experiment design is useful when several experiments can be run in parallel. Instead of designing one experiment at a time, we optimize multiple experiments together so that they provide complementary information about the model parameters. In Pyomo.DoE, this is done by maximizing a scalar function of the sum of the Fisher information matrices from each experiment, together with any prior information.

Please open the notebook in Colab and run the first cell (takes a few minutes).

Import Necessary Functions for the Two-State TC Lab Model

Load Experimental Data (sine test)

Use Prior Parameter Information

Initialize Two Experiments

Optimize Two Experiments (D-Optimality)

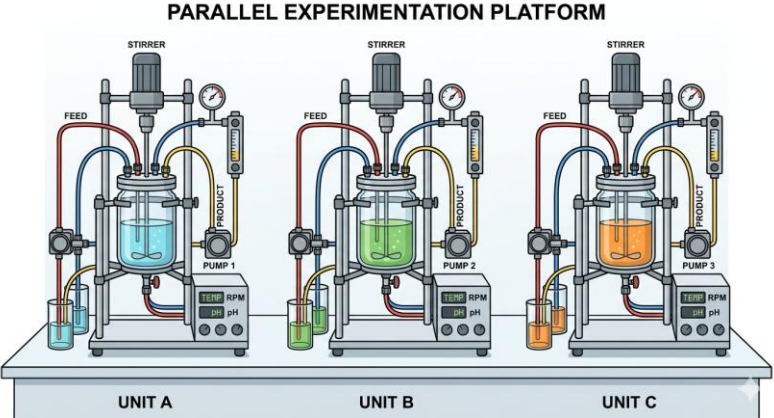
Compare the Reduction in Predicted Uncertainty

Activity: Repeat the Design with A-Optimality

Analyze the Results

Why design multiple experiments?

Multiple experimental setups¹



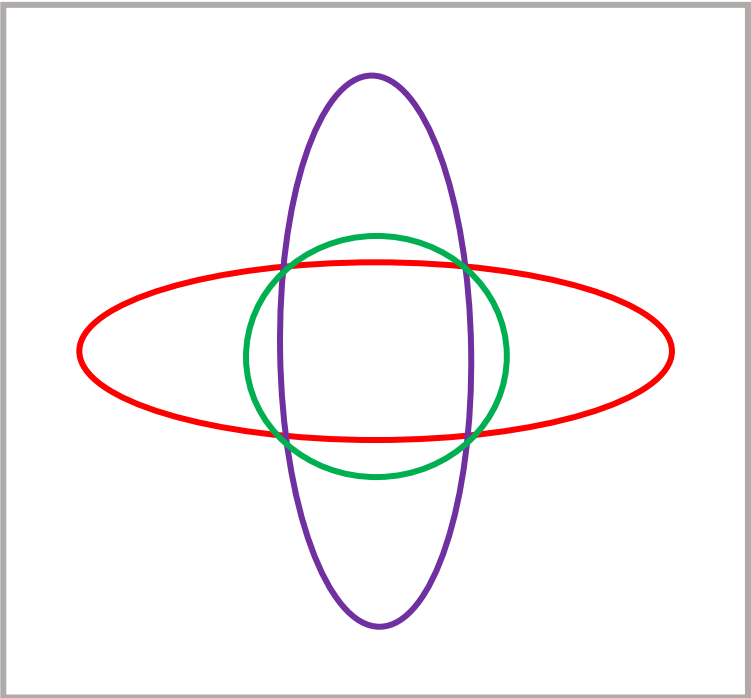
Gemini-generated image



<https://ucallmlabs.com>

Complementary Information²

Parameter 2



Parameter 1

Confidence ellipse:

- exp 1 only
- exp 2 only
- exp 1 and 2 combined

[1] Galvanin et al. (2007), *Industrial & Eng. Chem. Research*

[2] Ballan et al. (2012), *Computer Aided Chem. Eng.*

Multi-experiment design formulation

$$\max_{\boldsymbol{\phi}_1, \dots, \boldsymbol{\phi}_{N_{\text{exp}}}} \psi(\mathbf{M})$$

Choose all experiment designs simultaneously

$$\text{s.t. } \mathbf{M} = \mathbf{M}_0 + \sum_{k \in E} \mathbf{M}_k(\hat{\boldsymbol{\theta}}, \boldsymbol{\phi}_k)$$

Combine information

$$\mathbf{M}_k(\hat{\boldsymbol{\theta}}, \boldsymbol{\phi}_k) = \mathbf{Q}_k^T \Sigma_y^{-1} \mathbf{Q}_k; \quad \forall k \in E$$

Information from experiment k

$$\mathbf{Q}_k(\hat{\boldsymbol{\theta}}) = \frac{\partial \mathbf{h}(\mathbf{x}_k(\mathbf{t}), \mathbf{z}_k(\mathbf{t}), \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \bigg|_{\hat{\boldsymbol{\theta}}}; \quad \forall k \in E$$

Sensitivity to parameters

$$\phi_{\text{primary},1} \leq \dots \leq \phi_{\text{primary},N_{\text{exp}}}$$

Avoid equivalent experiment permutations

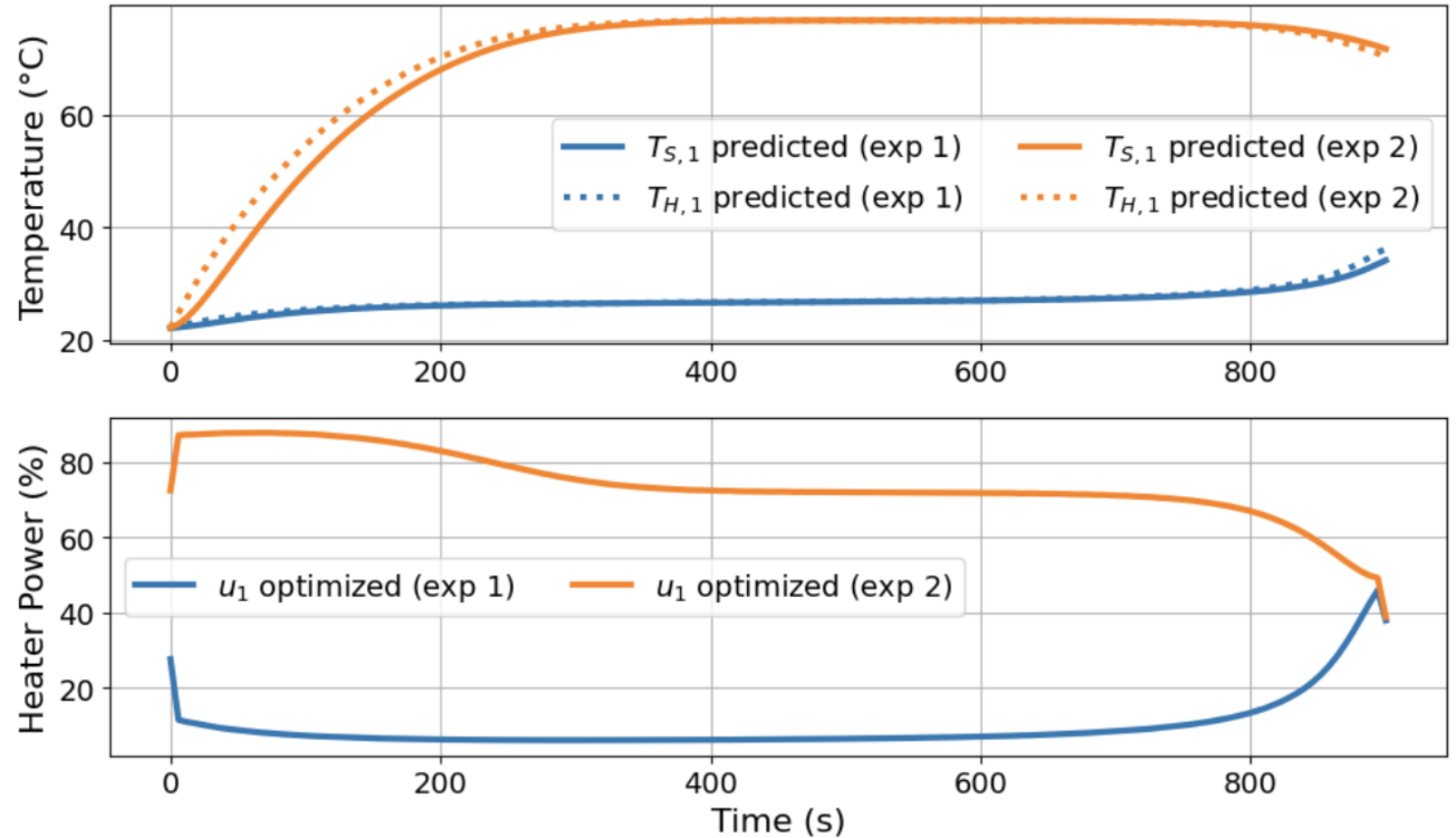
Each experiment contributes information; multi-experiment design optimizes the total information from the set

E : Set of experiments, $\{1, \dots, N_{\text{exp}}\}$; $\mathbf{M}$: Fisher information matrix; $\mathbf{M}_0$: Prior Fisher information matrix; $\hat{\mathbf{y}}_k$: Model prediction for experiment k; $\hat{\boldsymbol{\theta}}$: Parameter estimate; $\boldsymbol{\phi}$: Design vector; ϕ_{primary} : A selected variable from the design vector $\boldsymbol{\phi}$

Galvanin et al. (2007), Industrial & Eng. Chem. Research

TC Lab: Multi-experiment Design with D-optimality

$$\begin{aligned}
 & \max_{u_1, \dots, u_{N_{\text{exp}}}} \psi \left(\mathbf{M}_0 + \sum_{k \in E} \mathbf{M}_k(u_k) \right) \\
 & \text{s. t. } C_p^H \frac{dT_{H,k}}{dt} = U_a(T_{\text{amb},k} - T_{H,k}) + \dots \\
 & C_p^S \frac{dT_{S,k}}{dt} = U_b(T_{H,k} - T_{S,k}); \quad \forall k \in E \\
 & 0\% \leq u_k(t) \leq 100\%; \quad \forall k \in E \\
 & T_{H,k}(t_0) = T_{\text{amb},k}; \quad \forall k \in E \\
 & T_{S,k}(t_0) = T_{\text{amb},k}; \quad \forall k \in E
 \end{aligned}$$



Exp 1 yields information along the coordinate axes of internal parameters (U_b , C_p^S , C_p^H). Exp 2 yields information along the axis of ambient dissipation (U_a).

Getting Started with Pyomo.DoE

Documentation: https://pyomo.readthedocs.io/en/stable/contributed_packages/doe/doe.html

Community Detection for Pyomo models

Pyomo.DoE

Methodology Overview

Pyomo.DoE Required Inputs

Pyomo.DoE Solver Interface

Pyomo.DoE Usage Example

GDPopt logic-based solver

Infeasible Irreducible System (IIS) Tool

Incidence Analysis

MindtPy Solver

MPC

Multistart Solver

Nonlinear Preprocessing Transformations

Parameter Estimation with parmest

PyNumero

PyROS Solver

Sensitivity Toolbox

Trust Region Framework Method Solver

MC++ Interface

z3 SMT Sat Solver Interface

Read the Docs

v: stable

/ Third-Party Contributions / Pyomo.DoE

Edit on GitHub

Pyomo.DoE

Pyomo.DoE (Pyomo Design of Experiments) is a Python library for model-based design of experiments using science-based models.

Pyomo.DoE was developed by **Jialu Wang** and **Alexander W. Dowling** at the University of Notre Dame as part of the **Carbon Capture Simulation for Industry Impact (CCSI2)** project, funded through the U.S. Department Of Energy Office of Fossil Energy.

If you use Pyomo.DoE, please cite:

[Wang and Dowling, 2022] Wang, Jialu, and Alexander W. Dowling. "Pyomo.DOE: An open-source package for model-based design of experiments in Python." *AIChE Journal* 68.12 (2022): e17813. <https://doi.org/10.1002/aic.17813>

Methodology Overview

Model-based Design of Experiments (MBDoe) is a technique to maximize the information gain of experiments by directly using science-based models with physically meaningful parameters. It is one key component in the model calibration and uncertainty quantification workflow shown below:

```
graph LR; A([Prior knowledge, preliminary data]) --> B[Model]; B --> C[Exploratory analysis]; C --> D[Parameter estimation]; D --> E[Uncertainty analysis]; E --> F([Model with quantified uncertainty]); C --> B; E --> B; E --> C;
```


Notebooks at dowlinglab.github.io/pyomo-doe

Exercise: Pyomo.DoE

Alexander Dowling, Dan Laky, Shammah Lilonfe, Stephen Cini, Shuvashish Mondal,
Shilpa Narasimhan

In this notebook, you will use Pyomo.DoE to compute the A- and D-optimal experiments from the TCLab. In our [previous notebook](#), we used the sine test as a starting point. We will do the same here as well.

Recall, we can computing the next best experiment assuming we already completed one prior experiment. Thus it is important to confirm our optimal experiment design does not change if we change the prior experiment of optimization initial point.

```
import sys

# If running on Google Colab, install Pyomo and Ipopt via IDAES
on_colab = "google.colab" in sys.modules
if on_colab:
    !wget "https://raw.githubusercontent.com/dowlinglab/pyomo-doe/main/notebooks/tc
else:
    import os
```

CONTENTS ▾

- Load and Explore Experimental Data (sine test)
- Use Prior Parameter Information
- Optimize the Next Experiment (Pseudo-A-Optimality)
- Optimize the Next Experiment (D-Optimality)
- Optimize the Next Experiment (ME-Optimality)
- Exercise: Multi-experiment design
 - Load Experimental Data (sine test)
 - Use Prior Parameter Information
 - Initialize Two Experiments
 - Optimize Two Experiments (D-Optimality)

Acknowledgements

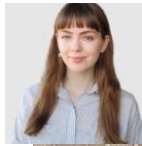
Pyomo.DoE development: *We graciously acknowledge funding from the U.S. Department of Energy, Office of Fossil Energy and Carbon Management, through the Carbon Capture Program (CCSI²) and Office of Resource Sustainability (PrOMMiS).*

This project was funded by the Department of Energy, National Energy Technology Laboratory an agency of the United States Government, through a support contract. Neither the United States Government nor any agency thereof, nor any of their employees, nor the support contractor, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.



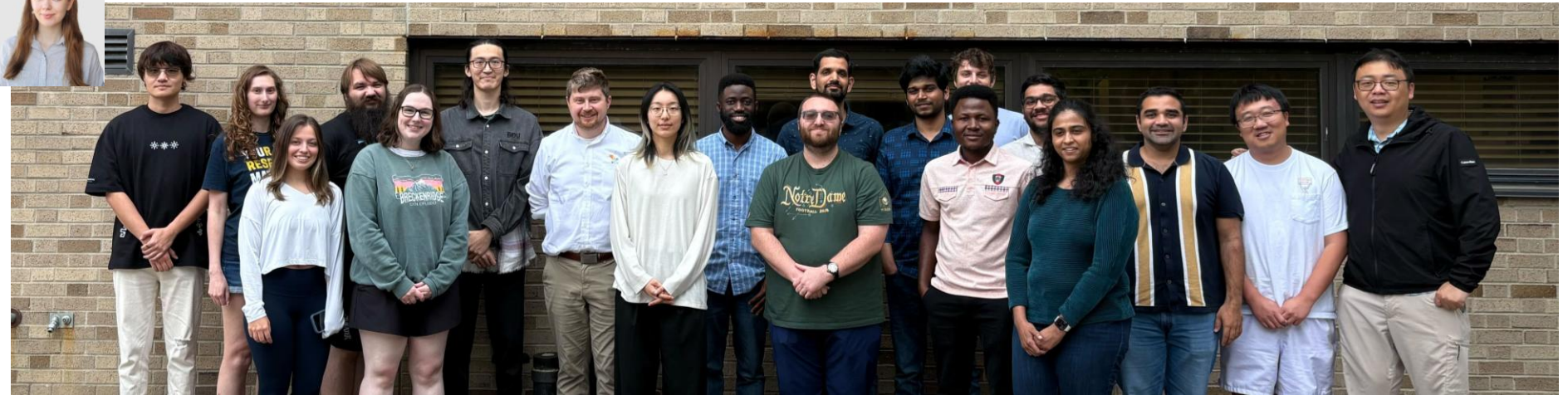
Pyomo Team (SNL):

- Bethany Nicholson
- John Sirola
- Miranda Mundt



Contributors (ND):

- Dr. Jialu Wang
- Hailey Lynch



Thank you to Prof. Jeff Kantor (1954-2023) for the TCLab examples and so much more.